

Appendix G: ResNet50V2 Tests 3

In []: `# I dont know if this was loading prev model so will have to do this all over ag`

In [1]: `!nvidia-smi`

```
Fri Mar 14 19:46:24 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15          CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|-----+-----+-----+-----+
| 0    N/A    45C    P0              58W / 400W |      0MiB / 40960MiB |           0%
Default |
|-----+-----+-----+-----+
| Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|-----+-----+-----+-----+
|      ID    ID
age      |
+-----+-----+-----+-----+
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`

```

from tensorflow.keras.applications import ResNet50 # Access to pre-trained model
from tensorflow.keras import layers, models, optimizers # Access to building blo
from tensorflow.keras.models import load_model # Ability to Load best_saved mode
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLRO
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis o
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Function definitions

```

In [4]: def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224)) # Resize images
    x = tf.image.random_flip_left_right(x) # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2) # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2) # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32)) # One-hot en
    return x, y

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()

```

```
plt.grid()

# Plot Training and Validation Accuracy
plt.subplot(1, 2, 2)
plt.plot(epochs, train_acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

Dropout(0.6), weight_decay(3e-2)

```
In [6]: for repeat_2_times in range(1):
        ##### <<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_size=0.1)
        print(f"X_temp.shape: {X_temp.shape}\n")

        # Split temp data into equal validation (4000) and testing (4000) data
        X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp, y_temp, test_size=0.5)
        print(f"X_temp_val.shape: {X_temp_val.shape}")
        print(f"y_temp_val.shape: {y_temp_val.shape}")
        print(f"X_temp_test.shape: {X_temp_test.shape}")
        print(f"y_temp_test.shape: {y_temp_test.shape}\n")

        # Split test data into validation (5000) and testing (5000)
        X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5)

        # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
        X_val = np.concatenate((X_val, X_temp_val), axis=0)
        y_val = np.concatenate((y_val, y_temp_val), axis=0)
        X_test = np.concatenate((X_test, X_temp_test), axis=0)
        y_test = np.concatenate((y_test, y_temp_test), axis=0)

        print(f"X_train.shape: {X_train.shape}")
        print(f"y_train.shape: {y_train.shape}")
        print(f"X_val.shape: {X_val.shape}")
        print(f"y_val.shape: {y_val.shape}")
        print(f"X_test.shape: {X_test.shape}")
        print(f"y_test.shape: {y_test.shape}\n")

        display_imgs(X_train, y_train)

        # Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
        X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
        display_imgs(X_train, y_train)

        labels_names = ['beaver', 'dolphins', 'otters', 'seals', 'whales', 'aquarium fish', 'flowers',
                        'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'fruit',
                        'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clocks']
```

```

        'telephone','television','bed','chair','couch','table','wardr
        'caterpillar','cockroach','bear','leopard','lion','tiger','wo
        'road','skyscraper','cloud','forest','mountain','plain','sea'
        'elephant','kangaroo','fox','porcupine','possum','raccoon','s
        'spider','worm','baby','boy','girl','man','woman','crocodile'
        'turtle','hamster','mouse','rabbit','shrew','squirrel','maple
        'bicycle','bus','motorcycle','pickup truck','train','lawn-mow
        'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                      tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                    tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can

```

```

resnet_50_base.trainable = False

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir='./Logs')
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model.h5', # Path where the best model will be saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune.h5', # Path where the best model will be saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss

```

```

        verbose=1) # Print out message when saving mode
#cvss_logger = CSVLogger('training_log.csv', separator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping, checkp

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model.h5')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

```

```

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
               loss='categorical_crossentropy',
               metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<Modify Dataset>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[early_stopping_monitor])

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune.h5')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores: {np.average(test_f1_scores.numpy()*100):.2f}%")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>

```

```

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

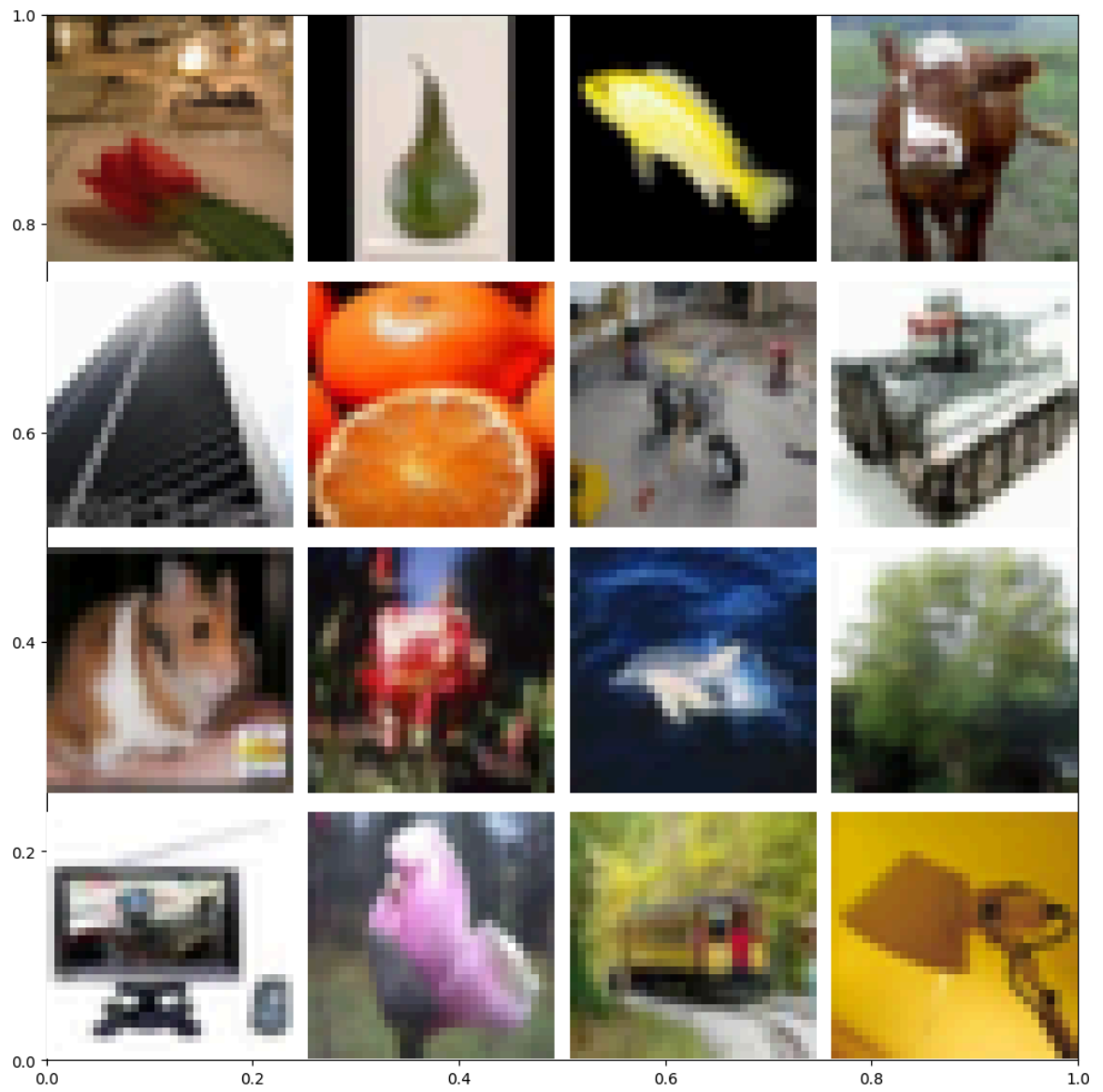
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

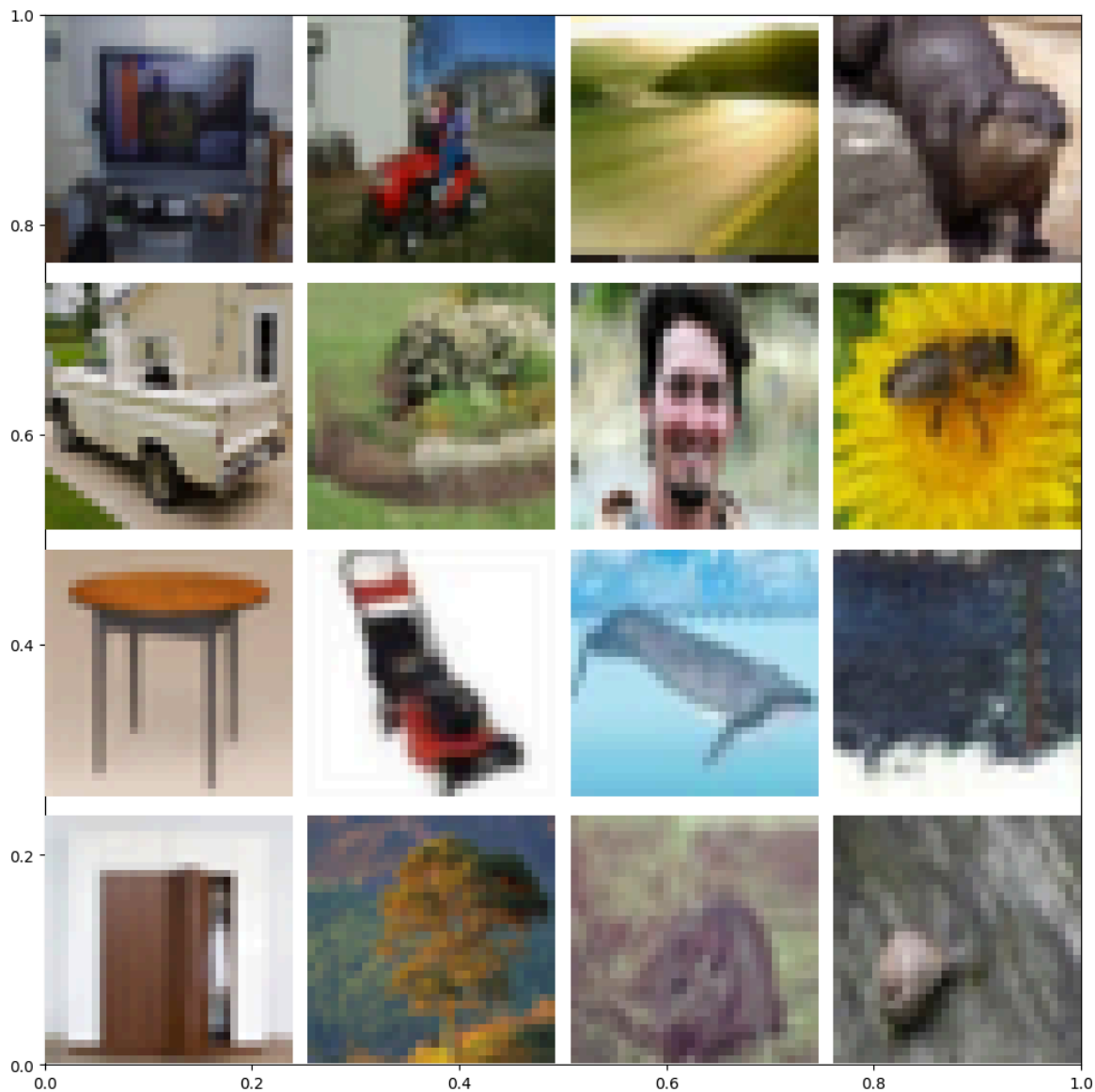
labels[15967].shape: (1,)

imgs[15967].shape: (32, 32, 3)



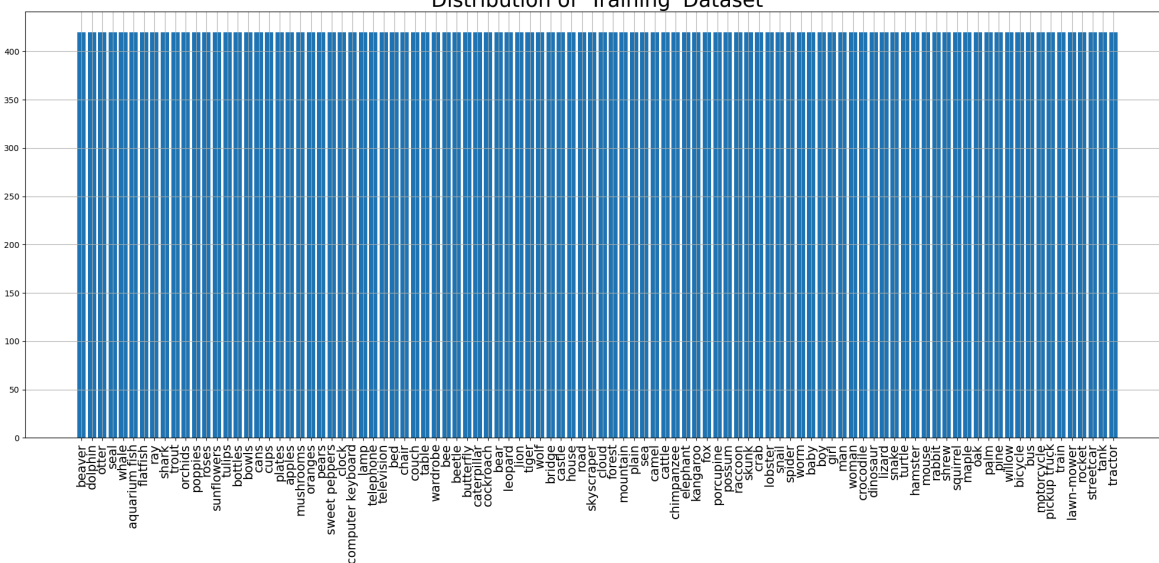
labels[20541].shape: (1,)

imgs[20541].shape: (32, 32, 3)

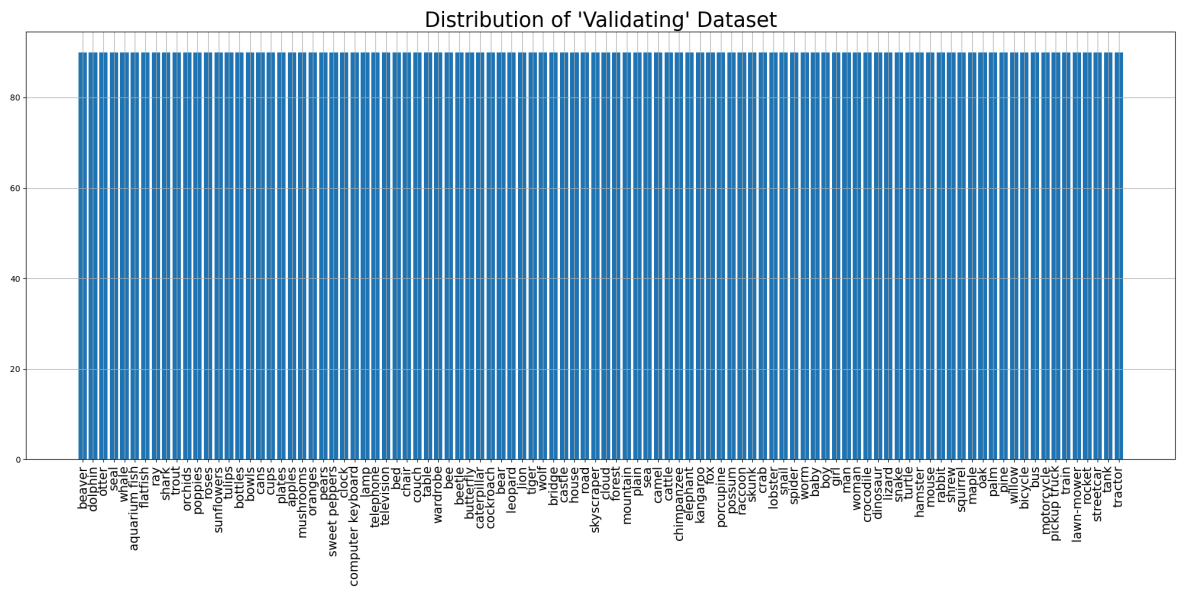


```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

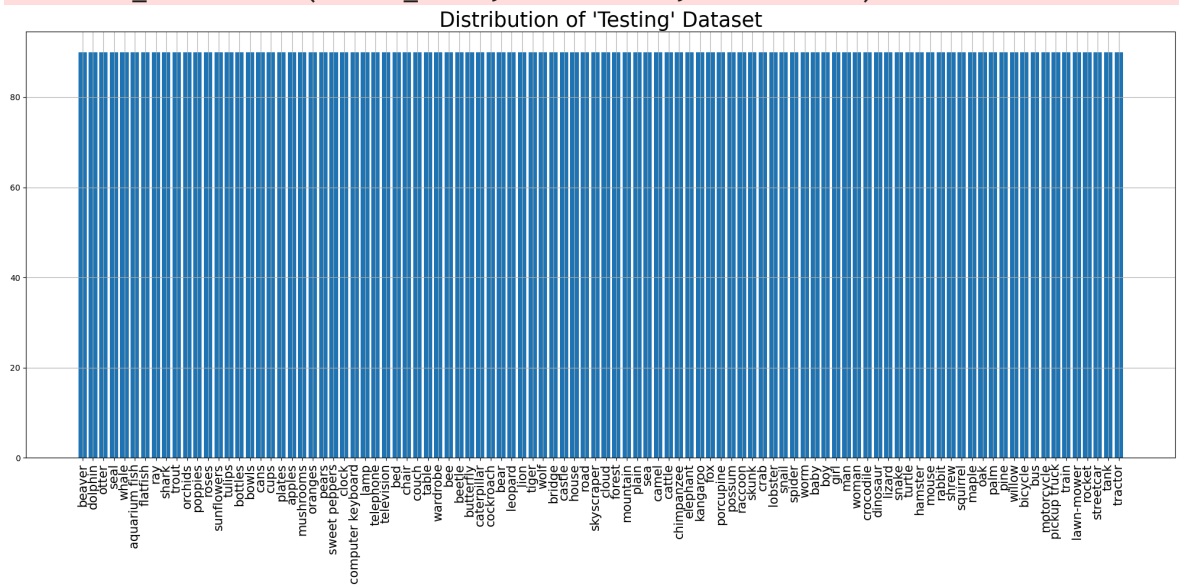
Distribution of 'Training' Dataset



```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657  **0s** 29ms/step - accuracy: 0.3164 - f1_score: 0.3116 - loss: 2.9046 - precision: 0.6484

Epoch 1: val_loss improved from inf to 1.51133, saving model to best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

657/657 ----- 40s 45ms/step - accuracy: 0.3166 - f1_score: 0.3117
- loss: 2.9037 - precision: 0.6485 - val_accuracy: 0.5856 - val_f1_score: 0.5789
- val_loss: 1.5113 - val_precision: 0.8287
Epoch 2/25
655/657 ----- 0s 23ms/step - accuracy: 0.5265 - f1_score: 0.5231 -
loss: 1.7430 - precision: 0.7450
Epoch 2: val_loss improved from 1.51133 to 1.40100, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5265 - f1_score: 0.5231
- loss: 1.7427 - precision: 0.7450 - val_accuracy: 0.6113 - val_f1_score: 0.6077
- val_loss: 1.4010 - val_precision: 0.8063
Epoch 3/25
655/657 ----- 0s 24ms/step - accuracy: 0.5723 - f1_score: 0.5688 -
loss: 1.5419 - precision: 0.7656
Epoch 3: val_loss improved from 1.40100 to 1.35169, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5724 - f1_score: 0.5689
- loss: 1.5417 - precision: 0.7657 - val_accuracy: 0.6229 - val_f1_score: 0.6201
- val_loss: 1.3517 - val_precision: 0.8093
Epoch 4/25
656/657 ----- 0s 24ms/step - accuracy: 0.5944 - f1_score: 0.5907 -
loss: 1.4290 - precision: 0.7723
Epoch 4: val_loss improved from 1.35169 to 1.32579, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5944 - f1_score: 0.5907
- loss: 1.4289 - precision: 0.7723 - val_accuracy: 0.6250 - val_f1_score: 0.6243
- val_loss: 1.3258 - val_precision: 0.8035
Epoch 5/25
657/657 ----- 0s 23ms/step - accuracy: 0.6132 - f1_score: 0.6099 -
loss: 1.3541 - precision: 0.7850
Epoch 5: val_loss improved from 1.32579 to 1.31782, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

#####
#####
657/657 ----- 19s 29ms/step - accuracy: 0.6132 - f1_score: 0.6099
- loss: 1.3541 - precision: 0.7850 - val_accuracy: 0.6288 - val_f1_score: 0.6279
- val_loss: 1.3178 - val_precision: 0.8000
Epoch 6/25
655/657 ----- 0s 23ms/step - accuracy: 0.6262 - f1_score: 0.6235 -
loss: 1.2847 - precision: 0.7927
Epoch 6: val_loss improved from 1.31782 to 1.31131, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
#####
#####
657/657 ----- 19s 29ms/step - accuracy: 0.6262 - f1_score: 0.6235
- loss: 1.2846 - precision: 0.7927 - val_accuracy: 0.6311 - val_f1_score: 0.6304
- val_loss: 1.3113 - val_precision: 0.7947
Epoch 7/25
655/657 ----- 0s 23ms/step - accuracy: 0.6469 - f1_score: 0.6444 -
loss: 1.2122 - precision: 0.7980
Epoch 7: val_loss did not improve from 1.31131
657/657 ----- 18s 28ms/step - accuracy: 0.6469 - f1_score: 0.6445
- loss: 1.2121 - precision: 0.7980 - val_accuracy: 0.6352 - val_f1_score: 0.6344
- val_loss: 1.3235 - val_precision: 0.7841
Epoch 8/25
656/657 ----- 0s 23ms/step - accuracy: 0.6591 - f1_score: 0.6561 -
loss: 1.1695 - precision: 0.8016
Epoch 8: val_loss improved from 1.31131 to 1.30538, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

#####
#####

```

657/657 ————— **19s** 29ms/step - accuracy: 0.6591 - f1_score: 0.6561
 - loss: 1.1694 - precision: 0.8016 - val_accuracy: 0.6394 - val_f1_score: 0.6384
 - val_loss: 1.3054 - val_precision: 0.7907

Epoch 9/25

656/657 ————— **0s** 23ms/step - accuracy: 0.6664 - f1_score: 0.6633 -
 loss: 1.1274 - precision: 0.8036

Epoch 9: val_loss did not improve from 1.30538

657/657 ————— **18s** 28ms/step - accuracy: 0.6664 - f1_score: 0.6634
 - loss: 1.1273 - precision: 0.8036 - val_accuracy: 0.6363 - val_f1_score: 0.6359
 - val_loss: 1.3329 - val_precision: 0.7800

Epoch 10/25

655/657 ————— **0s** 23ms/step - accuracy: 0.6790 - f1_score: 0.6765 -
 loss: 1.0662 - precision: 0.8110

Epoch 10: val_loss did not improve from 1.30538

657/657 ————— **18s** 28ms/step - accuracy: 0.6790 - f1_score: 0.6765
 - loss: 1.0661 - precision: 0.8110 - val_accuracy: 0.6459 - val_f1_score: 0.6458
 - val_loss: 1.3192 - val_precision: 0.7848

Epoch 11/25

656/657 ————— **0s** 23ms/step - accuracy: 0.6875 - f1_score: 0.6849 -
 loss: 1.0370 - precision: 0.8132

Epoch 11: val_loss did not improve from 1.30538

657/657 ————— **18s** 28ms/step - accuracy: 0.6875 - f1_score: 0.6849
 - loss: 1.0369 - precision: 0.8132 - val_accuracy: 0.6390 - val_f1_score: 0.6402
 - val_loss: 1.3207 - val_precision: 0.7789

Epoch 12/25

657/657 ————— **0s** 23ms/step - accuracy: 0.6946 - f1_score: 0.6924 -
 loss: 1.0069 - precision: 0.8163

Epoch 12: val_loss did not improve from 1.30538

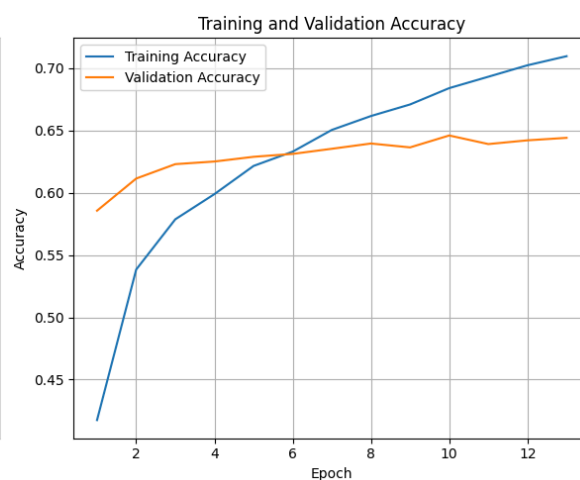
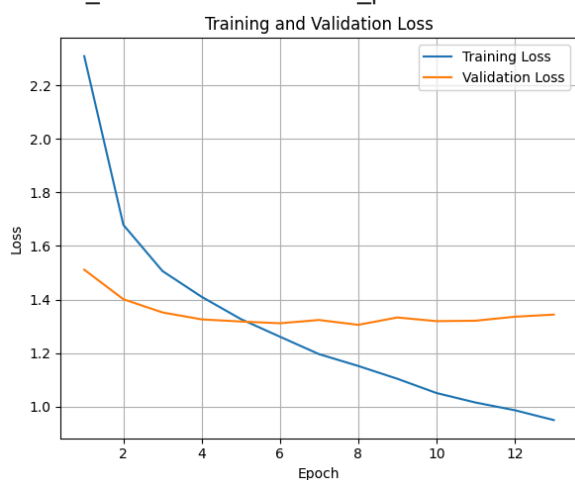
657/657 ————— **18s** 28ms/step - accuracy: 0.6946 - f1_score: 0.6924
 - loss: 1.0069 - precision: 0.8163 - val_accuracy: 0.6420 - val_f1_score: 0.6401
 - val_loss: 1.3357 - val_precision: 0.7732

Epoch 13/25

656/657 ————— **0s** 23ms/step - accuracy: 0.7042 - f1_score: 0.7021 -
 loss: 0.9666 - precision: 0.8215

Epoch 13: val_loss did not improve from 1.30538

657/657 ————— **18s** 28ms/step - accuracy: 0.7042 - f1_score: 0.7021
 - loss: 0.9666 - precision: 0.8215 - val_accuracy: 0.6440 - val_f1_score: 0.6434
 - val_loss: 1.3436 - val_precision: 0.7713



WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

141/141  **11s** 40ms/step - accuracy: 0.6259 - f1_score: 0.6181
- loss: 1.3250 - precision_3: 0.7790

Test Accuracy: 63.88%

Test Loss: 1.297981858253479

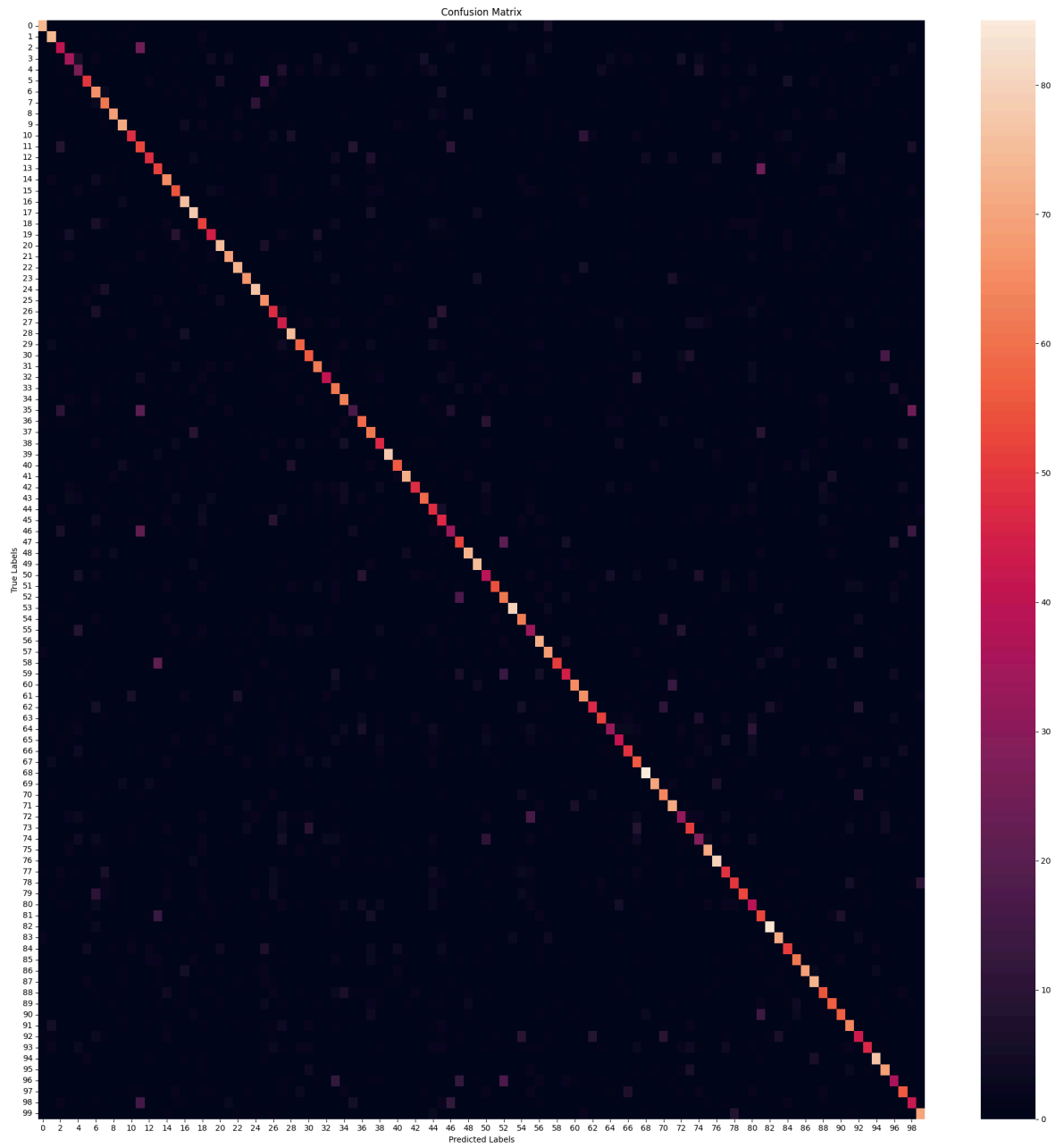
Test Precision: 78.91%

Test F1 Scores (Per Class): [86.04651 77.48691 46.242767 44.871788 31.515144 64.51612 56.89655

67.03297	78.65168	80.22598	61.53846	44.53781	61.53846	54.736835
67.01031	61.111103	77.15736	80.	54.450256	55.214714	80.21389
75.82417	80.66298	77.906975	80.628265	63.15789	48.979588	44.670044
79.787224	65.16853	63.276833	68.13186	53.947365	53.043472	65.62499
24.999996	67.04545	58.09523	54.97075	90.17341	67.46988	82.285706
64.42953	70.658676	46.601936	46.078426	39.75903	56.216217	80.66298
80.21389	41.711224	71.05263	59.512184	89.385475	65.957436	36.871502
73.846146	70.10309	71.32867	57.324833	77.38094	72.52747	61.03895
66.242035	44.13792	49.69696	59.39393	55.44554	89.00522	83.83234
67.36841	72.44897	38.509315	58.620686	33.93939	76.756744	82.47422
54.143642	62.1118	67.948715	38.80597	49.302315	89.361694	73.95833
59.30232	70.93022	79.768776	79.78141	60.773476	63.33333	62.63736
67.36841	49.142857	59.11949	86.666664	69.69697	42.857136	63.636356
42.424236	77.77777	]				

Average Test F1 Scores:63.78

282/282  **10s** 26ms/step



	precision	recall	f1-score	support
beaver	0.90	0.82	0.86	90
dolphin	0.73	0.82	0.77	90
otter	0.48	0.44	0.46	90
seal	0.53	0.39	0.45	90
whale	0.35	0.29	0.32	90
aquarium fish	0.77	0.56	0.65	90
flatfish	0.46	0.73	0.57	90
ray	0.66	0.68	0.67	90
shark	0.79	0.77	0.78	90
trout	0.82	0.79	0.80	90
orchids	0.73	0.53	0.62	90
poppies	0.36	0.59	0.45	90
roses	0.72	0.53	0.61	90
sunflowers	0.52	0.58	0.55	90
tulips	0.62	0.72	0.67	90
bottles	0.61	0.61	0.61	90
bowls	0.71	0.84	0.77	90
cans	0.74	0.87	0.80	90
cups	0.51	0.58	0.54	90
plates	0.62	0.50	0.55	90
apples	0.77	0.83	0.80	90
mushrooms	0.75	0.77	0.76	90
oranges	0.80	0.81	0.81	90
pears	0.82	0.74	0.78	90
sweet peppers	0.76	0.86	0.81	90
clock	0.55	0.73	0.63	90
computer keyboard	0.45	0.53	0.49	90
lamp	0.41	0.49	0.45	90
telephone	0.77	0.83	0.80	90
television	0.65	0.64	0.65	90
bed	0.64	0.62	0.63	90
chair	0.67	0.69	0.68	90
couch	0.66	0.46	0.54	90
table	0.44	0.68	0.53	90
wardrobe	0.62	0.70	0.66	90
bee	0.42	0.18	0.25	90
beetle	0.69	0.66	0.67	90
butterfly	0.51	0.68	0.58	90
caterpillar	0.58	0.52	0.55	90
cockroach	0.94	0.87	0.90	90
bear	0.74	0.62	0.67	90
leopard	0.85	0.80	0.82	90
lion	0.81	0.53	0.64	90
tiger	0.77	0.66	0.71	90
wolf	0.41	0.53	0.47	90
bridge	0.41	0.52	0.46	90
castle	0.43	0.37	0.40	90
house	0.55	0.58	0.56	90
road	0.80	0.81	0.81	90
skyscraper	0.77	0.83	0.80	90
cloud	0.41	0.43	0.42	90
forest	0.86	0.60	0.71	90
mountain	0.53	0.68	0.60	90
plain	0.90	0.89	0.89	90
sea	0.63	0.69	0.66	90
camel	0.37	0.37	0.37	90
cattle	0.69	0.80	0.74	90
chimpanzee	0.65	0.76	0.70	90

elephant	0.96	0.57	0.71	90
kangaroo	0.67	0.50	0.57	90
fox	0.83	0.72	0.77	90
porcupine	0.72	0.73	0.73	90
possum	0.73	0.52	0.61	90
raccoon	0.78	0.58	0.66	90
skunk	0.58	0.36	0.44	90
crab	0.55	0.46	0.50	90
lobster	0.65	0.54	0.59	90
snail	0.50	0.62	0.55	90
spider	0.84	0.94	0.89	90
worm	0.91	0.78	0.84	90
baby	0.65	0.71	0.68	90
boy	0.67	0.79	0.72	90
girl	0.44	0.34	0.39	90
man	0.61	0.57	0.59	90
woman	0.37	0.31	0.34	90
crocodile	0.75	0.79	0.77	90
dinosaur	0.77	0.89	0.82	90
lizard	0.54	0.54	0.54	90
snake	0.70	0.56	0.62	90
turtle	0.80	0.59	0.68	90
hamster	0.35	0.43	0.39	90
mouse	0.42	0.59	0.49	90
rabbit	0.86	0.93	0.89	90
shrew	0.70	0.79	0.74	90
squirrel	0.62	0.57	0.59	90
maple	0.74	0.68	0.71	90
oak	0.83	0.77	0.80	90
palm	0.78	0.81	0.80	90
pine	0.60	0.61	0.61	90
willow	0.63	0.63	0.63	90
bicycle	0.62	0.63	0.63	90
bus	0.64	0.71	0.67	90
motorcycle	0.51	0.48	0.49	90
pickup truck	0.68	0.52	0.59	90
train	0.87	0.86	0.86	90
lawn-mower	0.64	0.77	0.70	90
rocket	0.46	0.40	0.43	90
streetcar	0.65	0.62	0.64	90
tank	0.39	0.47	0.42	90
tractor	0.78	0.78	0.78	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2048)
dense_2 (Dense)	(None, 512)
dropout_1 (Dropout)	(None, 512)
dense_3 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/15

1314/1314 ————— 0s 26ms/step - accuracy: 0.6793 - f1_score: 0.6772
- loss: 1.0816 - precision: 0.8191

Epoch 1: val_loss improved from inf to 1.26254, saving model to best_model_fine_tune.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

1314/1314 ————— 54s 34ms/step - accuracy: 0.6793 - f1_score: 0.6772 - loss: 1.0816 - precision: 0.8191 - val_accuracy: 0.6510 - val_f1_score: 0.6500 - val_loss: 1.2625 - val_precision: 0.7989

Epoch 2/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7034 - f1_score: 0.7015
- loss: 0.9855 - precision: 0.8346

Epoch 2: val_loss improved from 1.26254 to 1.25491, saving model to best_model_fine_tune.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

1314/1314 ————— 35s 26ms/step - accuracy: 0.7034 - f1_score: 0.7015 - loss: 0.9855 - precision: 0.8346 - val_accuracy: 0.6531 - val_f1_score: 0.6521 - val_loss: 1.2549 - val_precision: 0.7955

Epoch 3/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7111 - f1_score: 0.7092
- loss: 0.9633 - precision: 0.8368

Epoch 3: val_loss improved from 1.25491 to 1.25045, saving model to best_model_fine_tune.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

1314/1314 ----- 35s 27ms/step - accuracy: 0.7111 - f1_score: 0.709
2 - loss: 0.9633 - precision: 0.8368 - val_accuracy: 0.6533 - val_f1_score: 0.652
6 - val_loss: 1.2504 - val_precision: 0.7945
Epoch 4/15
1312/1314 ----- 0s 24ms/step - accuracy: 0.7163 - f1_score: 0.7141
- loss: 0.9461 - precision: 0.8446
Epoch 4: val_loss improved from 1.25045 to 1.24797, saving model to best_model_fi
ne_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7162 - f1_score: 0.714
1 - loss: 0.9461 - precision: 0.8446 - val_accuracy: 0.6536 - val_f1_score: 0.652
8 - val_loss: 1.2480 - val_precision: 0.7934
Epoch 5/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7205 - f1_score: 0.7185
- loss: 0.9292 - precision: 0.8422
Epoch 5: val_loss improved from 1.24797 to 1.24536, saving model to best_model_fi
ne_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7205 - f1_score: 0.718
5 - loss: 0.9292 - precision: 0.8422 - val_accuracy: 0.6549 - val_f1_score: 0.653
9 - val_loss: 1.2454 - val_precision: 0.7927
Epoch 6/15
1314/1314 ----- 0s 24ms/step - accuracy: 0.7256 - f1_score: 0.7237
- loss: 0.9164 - precision: 0.8471
Epoch 6: val_loss improved from 1.24536 to 1.24283, saving model to best_model_fi
ne_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7256 - f1_score: 0.723
7 - loss: 0.9164 - precision: 0.8471 - val_accuracy: 0.6560 - val_f1_score: 0.655
2 - val_loss: 1.2428 - val_precision: 0.7959
Epoch 7/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7272 - f1_score: 0.7251
- loss: 0.9061 - precision: 0.8498
Epoch 7: val_loss improved from 1.24283 to 1.24163, saving model to best_model_fi
ne_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

1314/1314 ----- 35s 26ms/step - accuracy: 0.7272 - f1_score: 0.725
1 - loss: 0.9061 - precision: 0.8497 - val_accuracy: 0.6561 - val_f1_score: 0.655
3 - val_loss: 1.2416 - val_precision: 0.7948
Epoch 8/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7296 - f1_score: 0.7278
- loss: 0.9017 - precision: 0.8489
Epoch 8: val_loss improved from 1.24163 to 1.23854, saving model to best_model_fi
ne_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7296 - f1_score: 0.727
8 - loss: 0.9018 - precision: 0.8489 - val_accuracy: 0.6569 - val_f1_score: 0.656
0 - val_loss: 1.2385 - val_precision: 0.7945
Epoch 9/15
1314/1314 ----- 0s 24ms/step - accuracy: 0.7289 - f1_score: 0.7271
- loss: 0.8905 - precision: 0.8514
Epoch 9: val_loss improved from 1.23854 to 1.23730, saving model to best_model_fi
ne_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7288 - f1_score: 0.727
1 - loss: 0.8905 - precision: 0.8514 - val_accuracy: 0.6568 - val_f1_score: 0.655
9 - val_loss: 1.2373 - val_precision: 0.7955
Epoch 10/15
1312/1314 ----- 0s 23ms/step - accuracy: 0.7319 - f1_score: 0.7299
- loss: 0.8892 - precision: 0.8539
Epoch 10: val_loss improved from 1.23730 to 1.23501, saving model to best_model_f
ine_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7319 - f1_score: 0.729
8 - loss: 0.8893 - precision: 0.8539 - val_accuracy: 0.6571 - val_f1_score: 0.656
3 - val_loss: 1.2350 - val_precision: 0.7962
Epoch 11/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7343 - f1_score: 0.7325
- loss: 0.8787 - precision: 0.8570
Epoch 11: val_loss improved from 1.23501 to 1.23360, saving model to best_model_f
ine_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

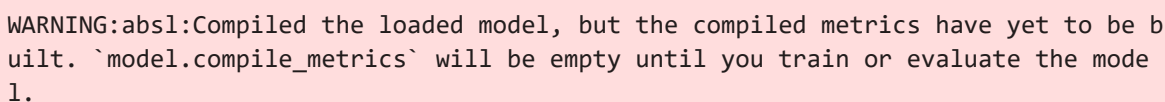
```

```

1314/1314 ----- 35s 26ms/step - accuracy: 0.7343 - f1_score: 0.732
5 - loss: 0.8787 - precision: 0.8569 - val_accuracy: 0.6569 - val_f1_score: 0.656
0 - val_loss: 1.2336 - val_precision: 0.7958
Epoch 12/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7361 - f1_score: 0.7346
- loss: 0.8740 - precision: 0.8573
Epoch 12: val_loss improved from 1.23360 to 1.23276, saving model to best_model_f
ine_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 27ms/step - accuracy: 0.7361 - f1_score: 0.734
6 - loss: 0.8740 - precision: 0.8573 - val_accuracy: 0.6578 - val_f1_score: 0.657
1 - val_loss: 1.2328 - val_precision: 0.7972
Epoch 13/15
1312/1314 ----- 0s 24ms/step - accuracy: 0.7373 - f1_score: 0.7355
- loss: 0.8682 - precision: 0.8563
Epoch 13: val_loss improved from 1.23276 to 1.23082, saving model to best_model_f
ine_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7373 - f1_score: 0.735
5 - loss: 0.8683 - precision: 0.8562 - val_accuracy: 0.6590 - val_f1_score: 0.657
9 - val_loss: 1.2308 - val_precision: 0.7963
Epoch 14/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7412 - f1_score: 0.7392
- loss: 0.8556 - precision: 0.8615
Epoch 14: val_loss improved from 1.23082 to 1.22935, saving model to best_model_f
ine_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
1314/1314 ----- 35s 26ms/step - accuracy: 0.7412 - f1_score: 0.739
2 - loss: 0.8557 - precision: 0.8615 - val_accuracy: 0.6592 - val_f1_score: 0.658
2 - val_loss: 1.2294 - val_precision: 0.7974
Epoch 15/15
1313/1314 ----- 0s 24ms/step - accuracy: 0.7395 - f1_score: 0.7378
- loss: 0.8528 - precision: 0.8598
Epoch 15: val_loss improved from 1.22935 to 1.22849, saving model to best_model_f
ine_tune.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

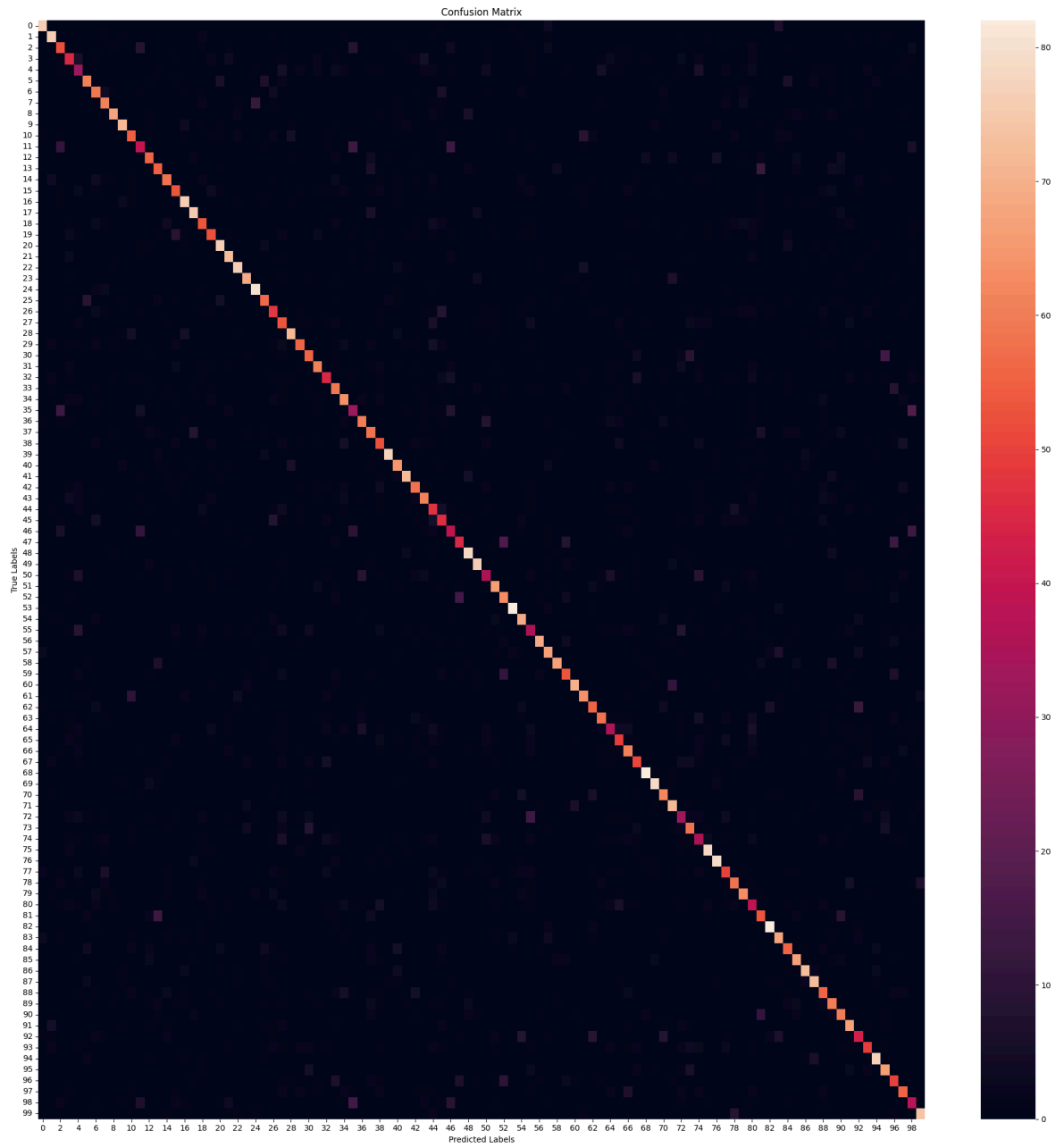
```
1314/1314  35s 26ms/step - accuracy: 0.7395 - f1_score: 0.737
8 - loss: 0.8528 - precision: 0.8598 - val_accuracy: 0.6598 - val_f1_score: 0.659
0 - val loss: 1.2285 - val precision: 0.7976
```



Test F1 Scores (Per Class): [84.91619 79.99999 55.319138 52.325577 35.555546 69.71428 61.458332

Average Test F1 Scores:66.48

24/75



	precision	recall	f1-score	support
beaver	0.85	0.84	0.85	90
dolphin	0.76	0.84	0.80	90
otter	0.53	0.58	0.55	90
seal	0.55	0.50	0.52	90
whale	0.36	0.36	0.36	90
aquarium fish	0.72	0.68	0.70	90
flatfish	0.58	0.66	0.61	90
ray	0.69	0.68	0.69	90
shark	0.86	0.76	0.80	90
trout	0.85	0.80	0.82	90
orchids	0.67	0.61	0.64	90
poppies	0.49	0.44	0.47	90
roses	0.67	0.62	0.64	90
sunflowers	0.62	0.61	0.62	90
tulips	0.72	0.64	0.68	90
bottles	0.62	0.59	0.60	90
bowls	0.75	0.84	0.80	90
cans	0.79	0.83	0.81	90
cups	0.63	0.59	0.61	90
plates	0.64	0.58	0.61	90
apples	0.78	0.84	0.81	90
mushrooms	0.74	0.81	0.78	90
oranges	0.83	0.84	0.84	90
pears	0.81	0.77	0.79	90
sweet peppers	0.79	0.89	0.84	90
clock	0.67	0.63	0.65	90
computer keyboard	0.55	0.53	0.54	90
lamp	0.45	0.59	0.51	90
telephone	0.82	0.80	0.81	90
television	0.75	0.62	0.68	90
bed	0.65	0.62	0.64	90
chair	0.75	0.68	0.71	90
couch	0.62	0.50	0.55	90
table	0.55	0.66	0.60	90
wardrobe	0.65	0.71	0.68	90
bee	0.38	0.36	0.37	90
beetle	0.71	0.67	0.69	90
butterfly	0.57	0.64	0.61	90
caterpillar	0.55	0.58	0.56	90
cockroach	0.94	0.86	0.90	90
bear	0.66	0.72	0.69	90
leopard	0.84	0.80	0.82	90
lion	0.66	0.64	0.65	90
tiger	0.71	0.68	0.69	90
wolf	0.42	0.52	0.47	90
bridge	0.46	0.51	0.49	90
castle	0.44	0.43	0.44	90
house	0.62	0.50	0.56	90
road	0.76	0.87	0.81	90
skyscraper	0.81	0.84	0.83	90
cloud	0.47	0.39	0.42	90
forest	0.73	0.73	0.73	90
mountain	0.61	0.70	0.65	90
plain	0.90	0.91	0.91	90
sea	0.67	0.77	0.72	90
camel	0.40	0.39	0.40	90
cattle	0.87	0.77	0.82	90
chimpanzee	0.71	0.74	0.73	90

elephant	0.87	0.74	0.80	90
kangaroo	0.65	0.59	0.62	90
fox	0.85	0.78	0.81	90
porcupine	0.76	0.71	0.74	90
possum	0.67	0.62	0.65	90
raccoon	0.73	0.64	0.69	90
skunk	0.52	0.40	0.45	90
crab	0.57	0.54	0.56	90
lobster	0.61	0.68	0.64	90
snail	0.62	0.57	0.59	90
spider	0.87	0.91	0.89	90
worm	0.82	0.88	0.85	90
baby	0.70	0.69	0.69	90
boy	0.71	0.80	0.75	90
girl	0.40	0.36	0.38	90
man	0.62	0.64	0.63	90
woman	0.40	0.39	0.39	90
crocodile	0.80	0.87	0.83	90
dinosaur	0.84	0.88	0.86	90
lizard	0.57	0.56	0.56	90
snake	0.71	0.64	0.67	90
turtle	0.72	0.70	0.71	90
hamster	0.39	0.43	0.41	90
mouse	0.55	0.59	0.57	90
rabbit	0.85	0.91	0.88	90
shrew	0.67	0.76	0.71	90
squirrel	0.64	0.62	0.63	90
maple	0.73	0.74	0.74	90
oak	0.82	0.81	0.82	90
palm	0.80	0.81	0.81	90
pine	0.61	0.62	0.62	90
willow	0.60	0.67	0.63	90
bicycle	0.61	0.67	0.64	90
bus	0.68	0.74	0.71	90
motorcycle	0.51	0.48	0.49	90
pickup truck	0.65	0.54	0.59	90
train	0.86	0.86	0.86	90
lawn-mower	0.65	0.73	0.69	90
rocket	0.51	0.56	0.53	90
streetcar	0.67	0.62	0.65	90
tank	0.43	0.42	0.42	90
tractor	0.80	0.82	0.81	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.66	9000
weighted avg	0.67	0.67	0.66	9000

Dropout(0.6), weight_decay(3e-2), l2(3e-2)

```
In [7]: for repeat_2_times in range(1):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

#             .map(augment_dataset, num_parallel_calls=tf.data.experimental.
#             .batch(batch_size)
#             .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-2) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model.h5', # Path where the best model will be saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

```

```

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model = load_model('best_model.h5')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>>

# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>

```

```

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune.h5')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores: {np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()

```



```
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

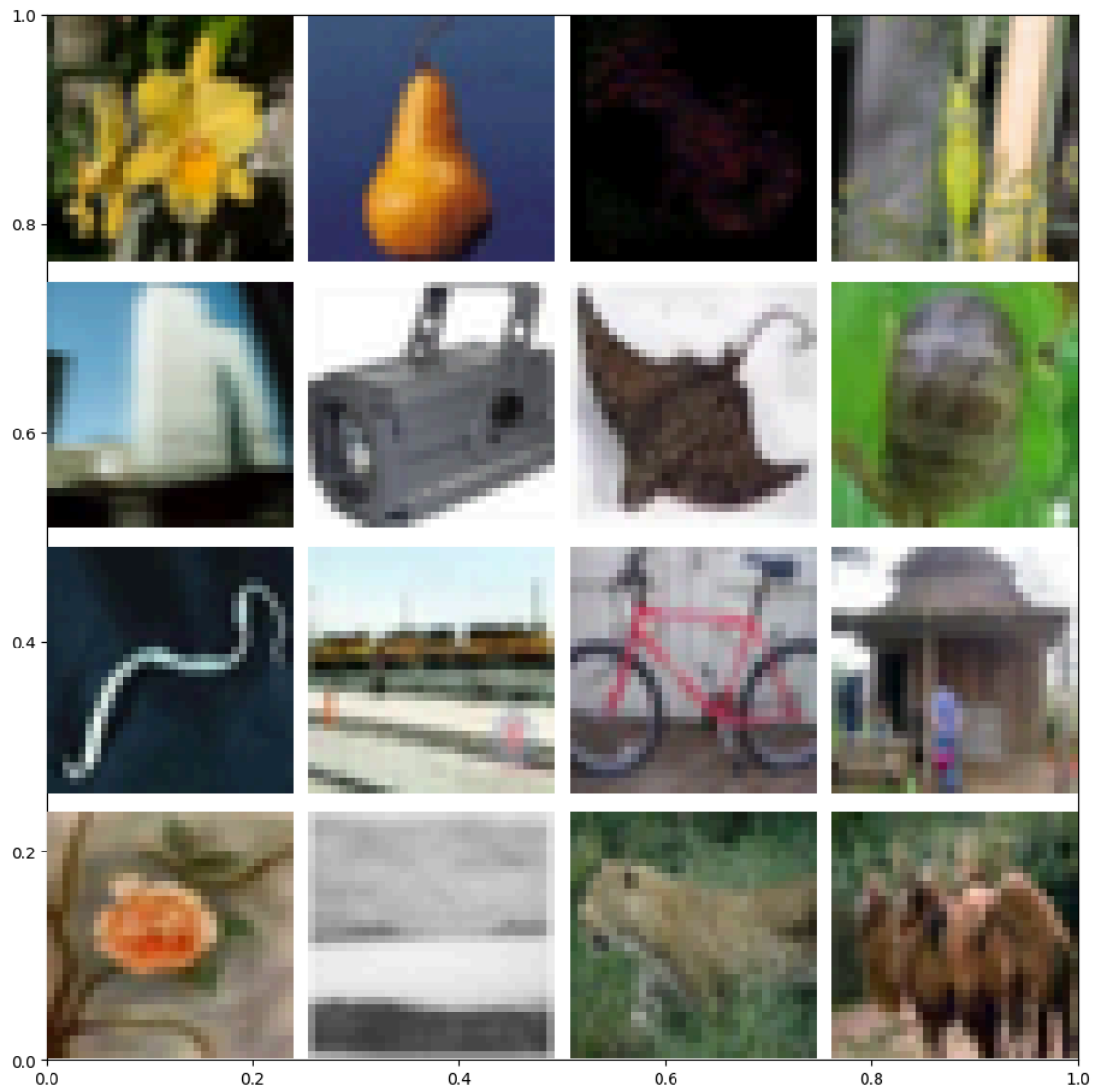
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

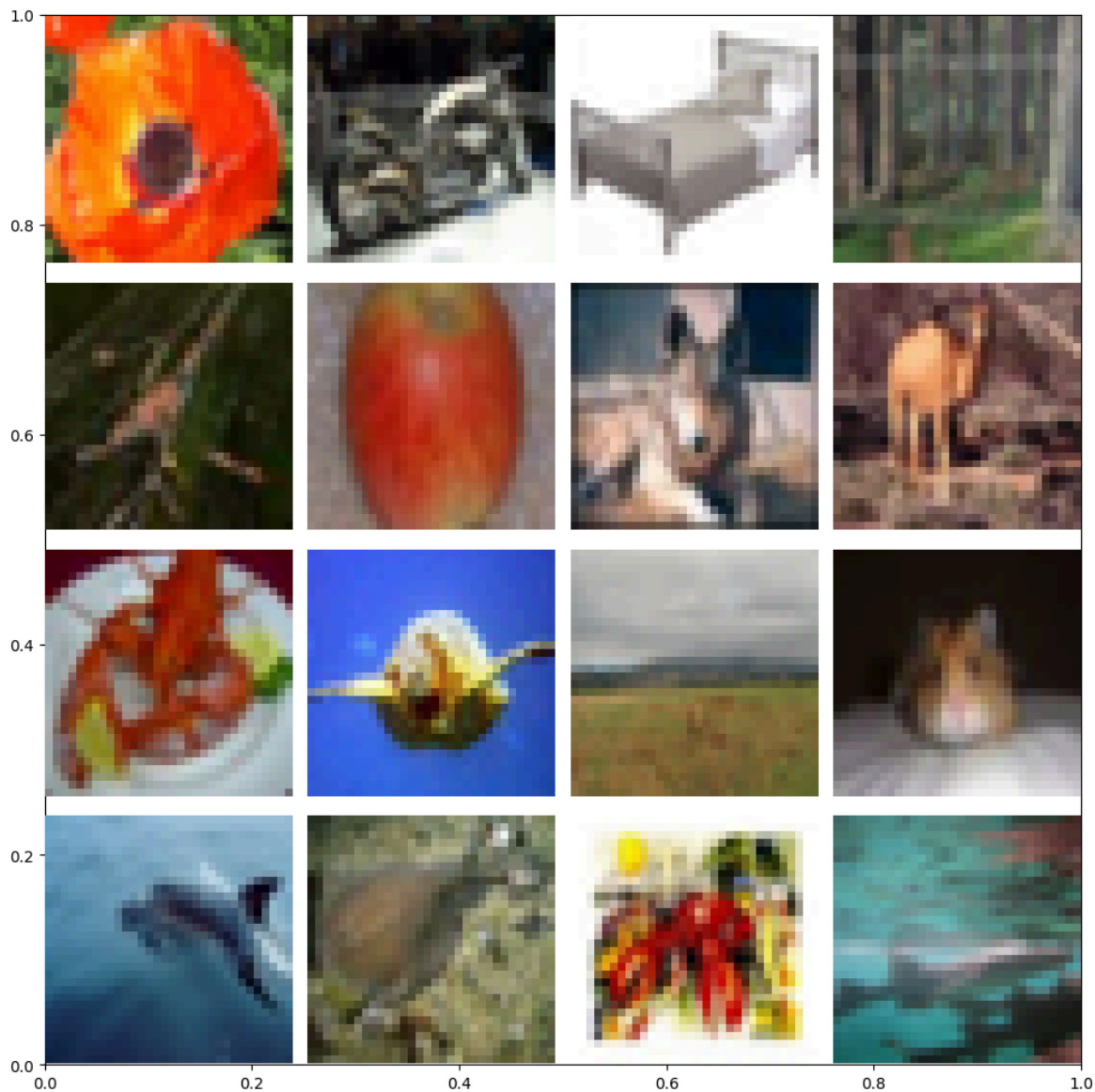
labels[6799].shape: (1,)

imgs[6799].shape: (32, 32, 3)



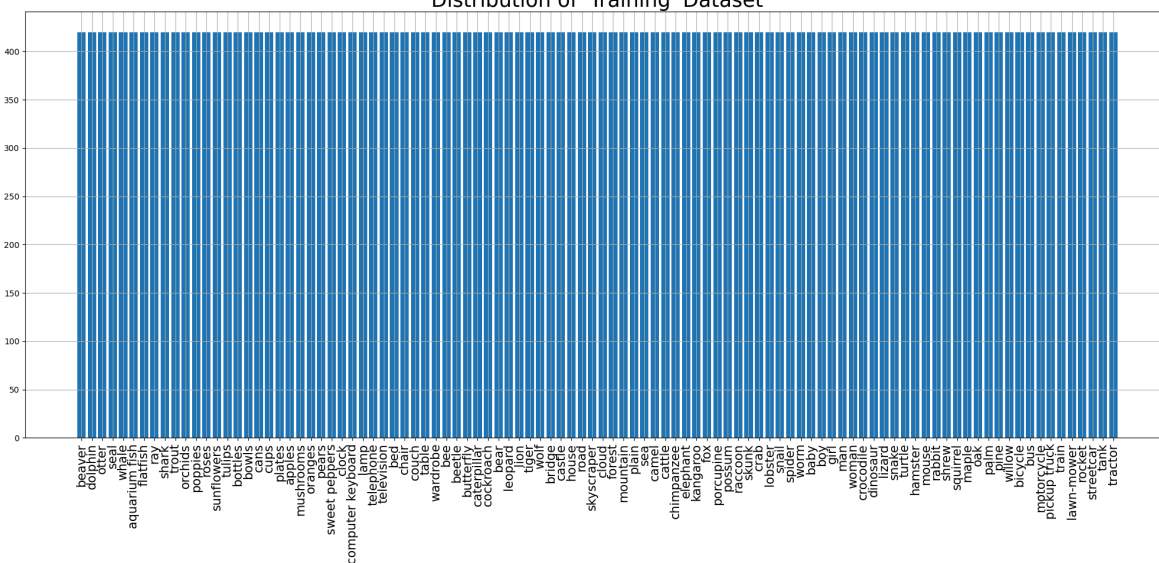
labels[41692].shape: (1,)

imgs[41692].shape: (32, 32, 3)

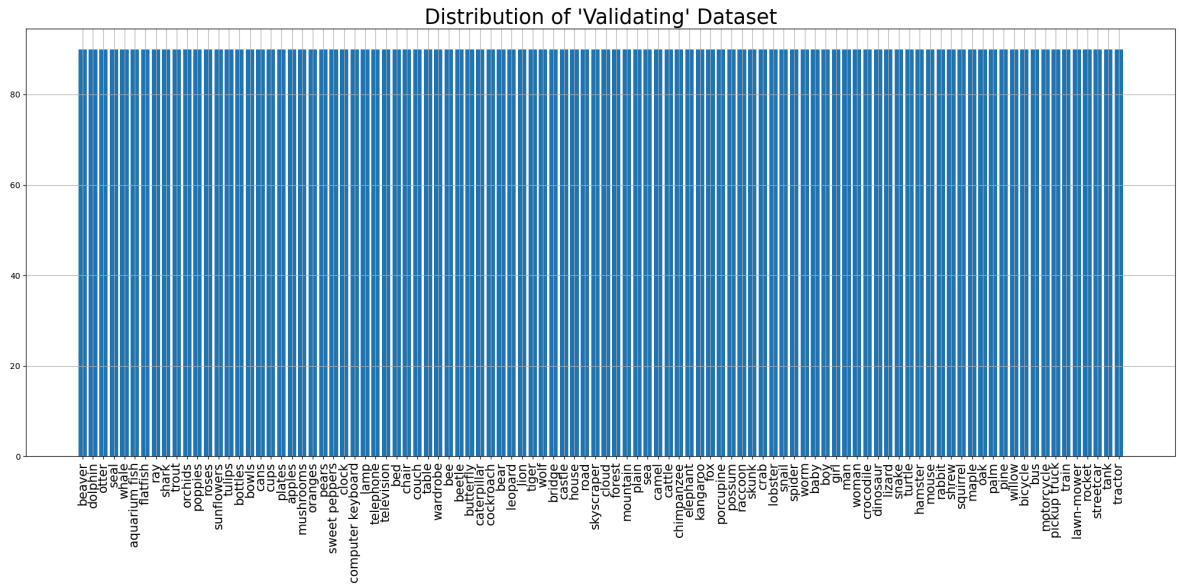


```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

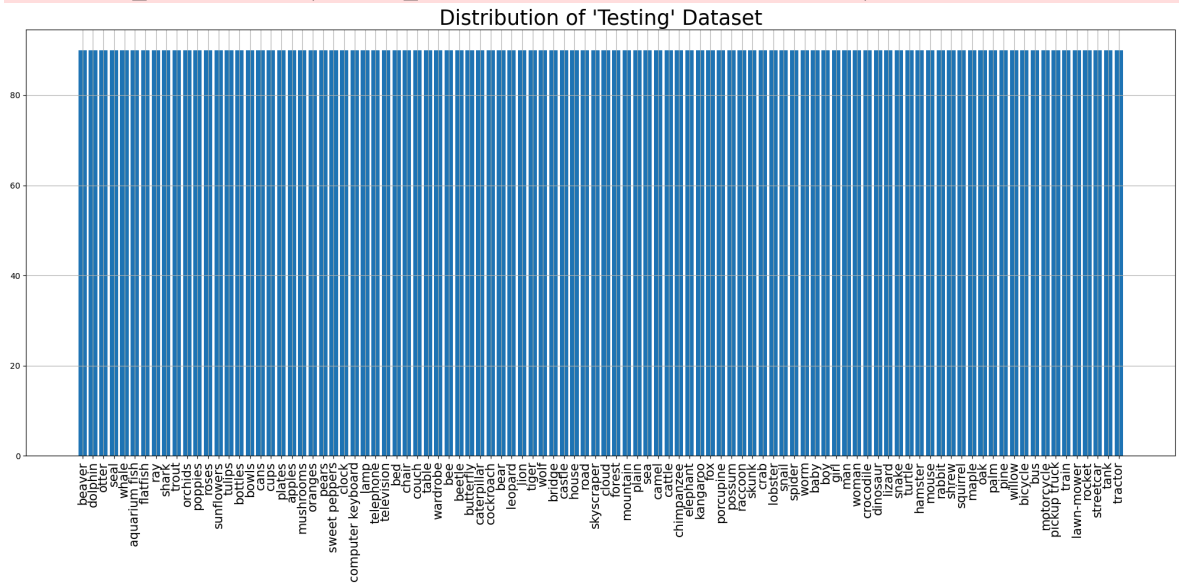
Distribution of 'Training' Dataset



```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657  **0s** 28ms/step - accuracy: 0.3151 - f1_score: 0.3097 - loss: 2.9192 - precision: 0.6598

Epoch 1: val_loss improved from inf to 1.50741, saving model to best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

657/657 ----- 40s 45ms/step - accuracy: 0.3152 - f1_score: 0.3099
- loss: 2.9183 - precision: 0.6599 - val_accuracy: 0.5861 - val_f1_score: 0.5841
- val_loss: 1.5074 - val_precision: 0.8201
Epoch 2/25
656/657 ----- 0s 23ms/step - accuracy: 0.5274 - f1_score: 0.5236 -
loss: 1.7341 - precision: 0.7457
Epoch 2: val_loss improved from 1.50741 to 1.40359, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5274 - f1_score: 0.5236
- loss: 1.7340 - precision: 0.7458 - val_accuracy: 0.6057 - val_f1_score: 0.6023
- val_loss: 1.4036 - val_precision: 0.8062
Epoch 3/25
657/657 ----- 0s 23ms/step - accuracy: 0.5648 - f1_score: 0.5620 -
loss: 1.5582 - precision: 0.7618
Epoch 3: val_loss improved from 1.40359 to 1.36195, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5648 - f1_score: 0.5621
- loss: 1.5581 - precision: 0.7618 - val_accuracy: 0.6214 - val_f1_score: 0.6188
- val_loss: 1.3620 - val_precision: 0.8029
Epoch 4/25
655/657 ----- 0s 23ms/step - accuracy: 0.5943 - f1_score: 0.5912 -
loss: 1.4389 - precision: 0.7751
Epoch 4: val_loss improved from 1.36195 to 1.34007, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5943 - f1_score: 0.5912
- loss: 1.4388 - precision: 0.7751 - val_accuracy: 0.6267 - val_f1_score: 0.6234
- val_loss: 1.3401 - val_precision: 0.8078
Epoch 5/25
656/657 ----- 0s 23ms/step - accuracy: 0.6110 - f1_score: 0.6078 -
loss: 1.3569 - precision: 0.7818
Epoch 5: val_loss improved from 1.34007 to 1.33104, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

657/657 ----- 19s 29ms/step - accuracy: 0.6111 - f1_score: 0.6079
- loss: 1.3568 - precision: 0.7818 - val_accuracy: 0.6309 - val_f1_score: 0.6283
- val_loss: 1.3310 - val_precision: 0.8016
Epoch 6/25
655/657 ----- 0s 23ms/step - accuracy: 0.6300 - f1_score: 0.6268 -
loss: 1.2746 - precision: 0.7901
Epoch 6: val_loss improved from 1.33104 to 1.32397, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.6300 - f1_score: 0.6268
- loss: 1.2745 - precision: 0.7901 - val_accuracy: 0.6329 - val_f1_score: 0.6318
- val_loss: 1.3240 - val_precision: 0.7901
Epoch 7/25
657/657 ----- 0s 23ms/step - accuracy: 0.6457 - f1_score: 0.6432 -
loss: 1.2133 - precision: 0.7962
Epoch 7: val_loss improved from 1.32397 to 1.31847, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.6458 - f1_score: 0.6432
- loss: 1.2133 - precision: 0.7962 - val_accuracy: 0.6310 - val_f1_score: 0.6303
- val_loss: 1.3185 - val_precision: 0.7913
Epoch 8/25
655/657 ----- 0s 23ms/step - accuracy: 0.6554 - f1_score: 0.6526 -
loss: 1.1705 - precision: 0.8003
Epoch 8: val_loss improved from 1.31847 to 1.30779, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.6554 - f1_score: 0.6527
- loss: 1.1704 - precision: 0.8003 - val_accuracy: 0.6370 - val_f1_score: 0.6347
- val_loss: 1.3078 - val_precision: 0.7849
Epoch 9/25
656/657 ----- 0s 23ms/step - accuracy: 0.6676 - f1_score: 0.6652 -
loss: 1.1133 - precision: 0.8083
Epoch 9: val_loss improved from 1.30779 to 1.29551, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

#####
#####

```

657/657 ————— 19s 29ms/step - accuracy: 0.6677 - f1_score: 0.6652
 - loss: 1.1132 - precision: 0.8083 - val_accuracy: 0.6456 - val_f1_score: 0.6445
 - val_loss: 1.2955 - val_precision: 0.7882

Epoch 10/25

656/657 ————— 0s 23ms/step - accuracy: 0.6777 - f1_score: 0.6746 -
 loss: 1.0717 - precision: 0.8074

Epoch 10: val_loss did not improve from 1.29551

657/657 ————— 18s 28ms/step - accuracy: 0.6777 - f1_score: 0.6747
 - loss: 1.0716 - precision: 0.8074 - val_accuracy: 0.6400 - val_f1_score: 0.6385
 - val_loss: 1.3152 - val_precision: 0.7763

Epoch 11/25

657/657 ————— 0s 23ms/step - accuracy: 0.6886 - f1_score: 0.6856 -
 loss: 1.0349 - precision: 0.8144

Epoch 11: val_loss did not improve from 1.29551

657/657 ————— 18s 28ms/step - accuracy: 0.6886 - f1_score: 0.6856
 - loss: 1.0349 - precision: 0.8144 - val_accuracy: 0.6418 - val_f1_score: 0.6412
 - val_loss: 1.3236 - val_precision: 0.7736

Epoch 12/25

655/657 ————— 0s 23ms/step - accuracy: 0.6949 - f1_score: 0.6927 -
 loss: 1.0036 - precision: 0.8179

Epoch 12: val_loss did not improve from 1.29551

657/657 ————— 18s 28ms/step - accuracy: 0.6950 - f1_score: 0.6927
 - loss: 1.0035 - precision: 0.8179 - val_accuracy: 0.6451 - val_f1_score: 0.6433
 - val_loss: 1.3343 - val_precision: 0.7757

Epoch 13/25

655/657 ————— 0s 23ms/step - accuracy: 0.7056 - f1_score: 0.7034 -
 loss: 0.9608 - precision: 0.8211

Epoch 13: val_loss did not improve from 1.29551

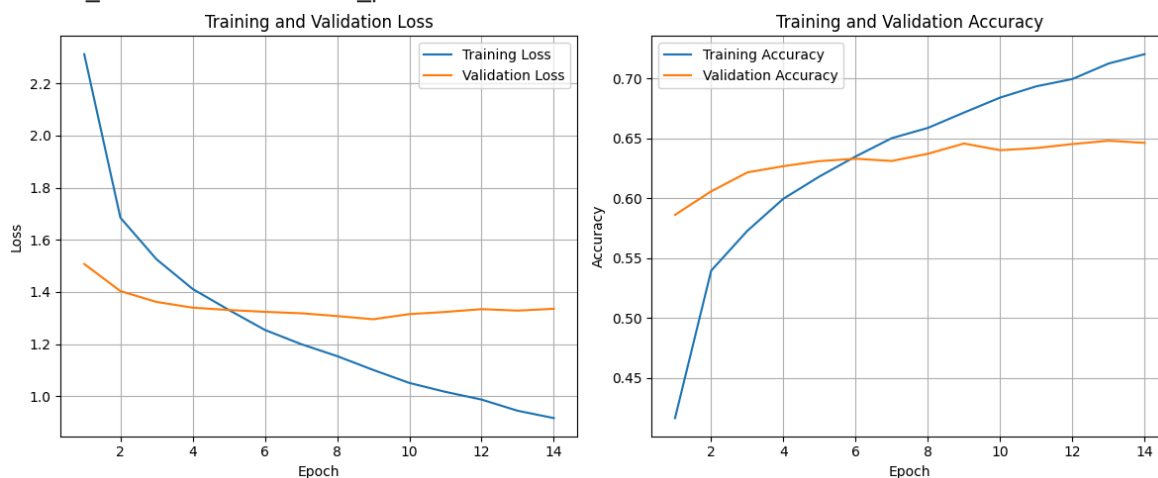
657/657 ————— 18s 28ms/step - accuracy: 0.7057 - f1_score: 0.7034
 - loss: 0.9607 - precision: 0.8212 - val_accuracy: 0.6479 - val_f1_score: 0.6463
 - val_loss: 1.3287 - val_precision: 0.7741

Epoch 14/25

657/657 ————— 0s 23ms/step - accuracy: 0.7136 - f1_score: 0.7114 -
 loss: 0.9378 - precision: 0.8265

Epoch 14: val_loss did not improve from 1.29551

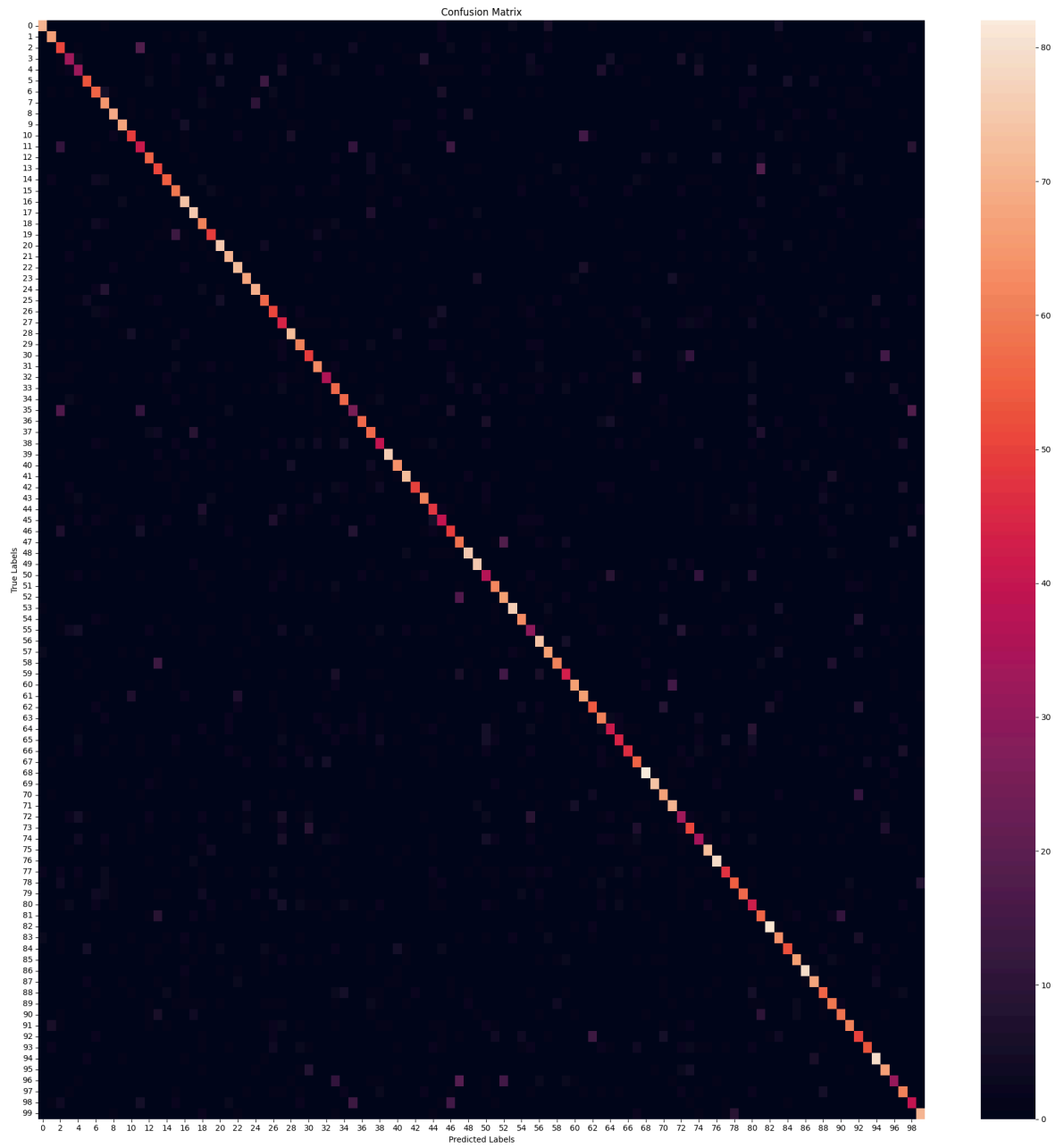
657/657 ————— 18s 28ms/step - accuracy: 0.7136 - f1_score: 0.7114
 - loss: 0.9378 - precision: 0.8265 - val_accuracy: 0.6461 - val_f1_score: 0.6436
 - val_loss: 1.3357 - val_precision: 0.7647



WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

141/141  **11s** 40ms/step - accuracy: 0.6366 - f1_score: 0.6288
- loss: 1.3285 - precision_7: 0.7839
Test Accuracy: 64.66%
Test Loss: 1.294567346572876
Test Precision: 79.08%
Test F1 Scores (Per Class): [83.5294 77.01149 51.256275 43.137253 39.024387 6
5.03067 61.202175
65.989845 79.31035 80.95238 62.025307 45.81005 64.36781 54.838707
67.07316 60.82474 76.68394 78.947365 59.512184 56.64739 82.41757
75.13227 79.347824 77.27273 79.54544 62.983418 54.838707 43.34975
81.56424 62.88659 55.999992 67.74193 49.315063 54.028427 65.517235
32.051277 71.69811 59.067356 47.61904 89.017334 69.565216 80.66298
67.11409 69.31818 53.631275 45.977005 50.7772 57.425735 82.872925
79.365074 41.111107 69.27374 64.07767 86.85714 69.23077 35.15151
69.81132 72.92817 75.77639 53.333324 76.300575 70.21276 61.714287
64.893616 49.10179 55.345905 62.585033 56.852787 84.97408 82.68156
70.526306 72.081215 37.714283 56.66666 38.857136 81.11111 82.7225
55.621304 67.48465 69.938644 37.16814 54.63414 89.01098 67.0157
62.275444 73.33333 88.26815 75.97765 61.538452 62.76595 62.702698
66.304344 52.631577 62.352936 83.76962 68.36735 41.610733 64.893616
45.197735 80.681816]
Average Test F1 Scores:64.57

282/282  **10s** 26ms/step



	precision	recall	f1-score	support
beaver	0.89	0.79	0.84	90
dolphin	0.80	0.74	0.77	90
otter	0.46	0.57	0.51	90
seal	0.52	0.37	0.43	90
whale	0.43	0.36	0.39	90
aquarium fish	0.73	0.59	0.65	90
flatfish	0.60	0.62	0.61	90
ray	0.61	0.72	0.66	90
shark	0.82	0.77	0.79	90
trout	0.87	0.76	0.81	90
orchids	0.72	0.54	0.62	90
poppies	0.46	0.46	0.46	90
roses	0.67	0.62	0.64	90
sunflowers	0.53	0.57	0.55	90
tulips	0.74	0.61	0.67	90
bottles	0.57	0.66	0.61	90
bowls	0.72	0.82	0.77	90
cans	0.75	0.83	0.79	90
cups	0.53	0.68	0.60	90
plates	0.59	0.54	0.57	90
apples	0.82	0.83	0.82	90
mushrooms	0.72	0.79	0.75	90
oranges	0.78	0.81	0.79	90
pears	0.78	0.76	0.77	90
sweet peppers	0.81	0.78	0.80	90
clock	0.63	0.63	0.63	90
computer keyboard	0.53	0.57	0.55	90
lamp	0.39	0.49	0.43	90
telephone	0.82	0.81	0.82	90
television	0.59	0.68	0.63	90
bed	0.58	0.54	0.56	90
chair	0.66	0.70	0.68	90
couch	0.64	0.40	0.49	90
table	0.47	0.63	0.54	90
wardrobe	0.68	0.63	0.66	90
bee	0.38	0.28	0.32	90
beetle	0.83	0.63	0.72	90
butterfly	0.56	0.63	0.59	90
caterpillar	0.51	0.44	0.48	90
cockroach	0.93	0.84	0.88	90
bear	0.68	0.71	0.70	90
leopard	0.80	0.81	0.81	90
lion	0.85	0.56	0.67	90
tiger	0.71	0.68	0.69	90
wolf	0.54	0.53	0.54	90
bridge	0.48	0.44	0.46	90
castle	0.48	0.54	0.51	90
house	0.52	0.64	0.57	90
road	0.82	0.83	0.83	90
skyscraper	0.76	0.83	0.79	90
cloud	0.41	0.41	0.41	90
forest	0.70	0.69	0.69	90
mountain	0.57	0.73	0.64	90
plain	0.89	0.84	0.87	90
sea	0.68	0.70	0.69	90
camel	0.39	0.32	0.35	90
cattle	0.60	0.82	0.69	90
chimpanzee	0.73	0.73	0.73	90

elephant	0.86	0.68	0.76	90
kangaroo	0.58	0.48	0.52	90
fox	0.80	0.73	0.76	90
porcupine	0.67	0.73	0.70	90
possum	0.64	0.60	0.62	90
raccoon	0.62	0.68	0.65	90
skunk	0.53	0.46	0.49	90
crab	0.64	0.49	0.55	90
lobster	0.81	0.51	0.63	90
snail	0.52	0.62	0.57	90
spider	0.80	0.91	0.85	90
worm	0.83	0.82	0.83	90
baby	0.67	0.74	0.71	90
boy	0.66	0.79	0.72	90
girl	0.39	0.37	0.38	90
man	0.57	0.57	0.57	90
woman	0.40	0.38	0.39	90
crocodile	0.81	0.81	0.81	90
dinosaur	0.78	0.88	0.83	90
lizard	0.59	0.52	0.56	90
snake	0.75	0.61	0.67	90
turtle	0.78	0.63	0.70	90
hamster	0.31	0.47	0.37	90
mouse	0.49	0.62	0.55	90
rabbit	0.88	0.90	0.89	90
shrew	0.63	0.71	0.67	90
squirrel	0.68	0.58	0.62	90
maple	0.73	0.73	0.73	90
oak	0.89	0.88	0.88	90
palm	0.76	0.76	0.76	90
pine	0.61	0.62	0.62	90
willow	0.60	0.66	0.63	90
bicycle	0.61	0.64	0.63	90
bus	0.65	0.68	0.66	90
motorcycle	0.50	0.56	0.53	90
pickup truck	0.66	0.59	0.62	90
train	0.80	0.88	0.84	90
lawn-mower	0.63	0.74	0.68	90
rocket	0.53	0.34	0.42	90
streetcar	0.62	0.68	0.65	90
tank	0.46	0.44	0.45	90
tractor	0.83	0.79	0.81	90
accuracy			0.65	9000
macro avg	0.65	0.65	0.65	9000
weighted avg	0.65	0.65	0.65	9000

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)



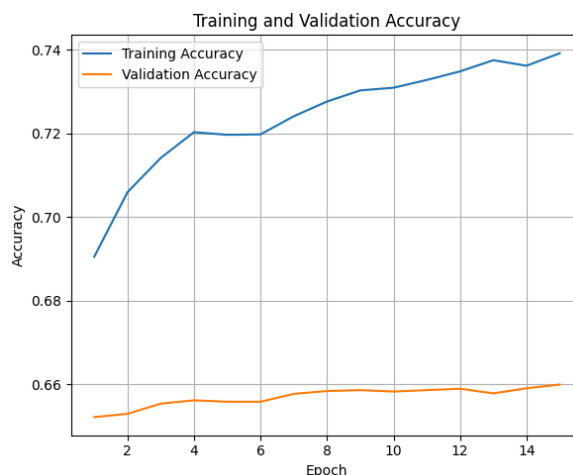
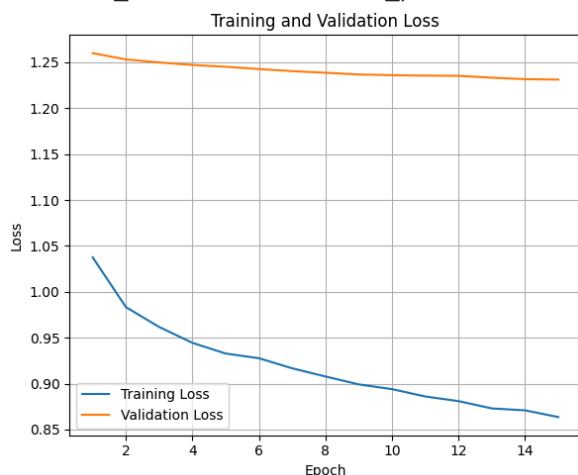
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1314/1314 ————— 0s 26ms/step - accuracy: 0.6910 - f1_score: 0.6891
- loss: 1.0244 - precision: 0.8222
Epoch 1: val_loss did not improve from 1.22849
1314/1314 ————— 54s 33ms/step - accuracy: 0.6910 - f1_score: 0.689
1 - loss: 1.0244 - precision: 0.8222 - val_accuracy: 0.6521 - val_f1_score: 0.650
8 - val_loss: 1.2600 - val_precision: 0.7968
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7130 - f1_score: 0.7112
- loss: 0.9502 - precision: 0.8374
Epoch 2: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7130 - f1_score: 0.711
2 - loss: 0.9502 - precision: 0.8374 - val_accuracy: 0.6529 - val_f1_score: 0.651
9 - val_loss: 1.2532 - val_precision: 0.7962
Epoch 3/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7199 - f1_score: 0.7181
- loss: 0.9281 - precision: 0.8408
Epoch 3: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7199 - f1_score: 0.718
1 - loss: 0.9282 - precision: 0.8408 - val_accuracy: 0.6553 - val_f1_score: 0.654
3 - val_loss: 1.2499 - val_precision: 0.7978
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7266 - f1_score: 0.7249
- loss: 0.9113 - precision: 0.8477
Epoch 4: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7266 - f1_score: 0.724
9 - loss: 0.9114 - precision: 0.8477 - val_accuracy: 0.6561 - val_f1_score: 0.655
0 - val_loss: 1.2471 - val_precision: 0.7961
Epoch 5/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7283 - f1_score: 0.7265
- loss: 0.8995 - precision: 0.8479
Epoch 5: val_loss did not improve from 1.22849
1314/1314 ————— 35s 26ms/step - accuracy: 0.7283 - f1_score: 0.726
5 - loss: 0.8995 - precision: 0.8479 - val_accuracy: 0.6558 - val_f1_score: 0.654
7 - val_loss: 1.2452 - val_precision: 0.7960
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7286 - f1_score: 0.7269
- loss: 0.8865 - precision: 0.8499
Epoch 6: val_loss did not improve from 1.22849
1314/1314 ————— 35s 26ms/step - accuracy: 0.7285 - f1_score: 0.726
9 - loss: 0.8866 - precision: 0.8499 - val_accuracy: 0.6558 - val_f1_score: 0.654
6 - val_loss: 1.2427 - val_precision: 0.7964
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7321 - f1_score: 0.7301
- loss: 0.8783 - precision: 0.8521
Epoch 7: val_loss did not improve from 1.22849
1314/1314 ————— 35s 26ms/step - accuracy: 0.7321 - f1_score: 0.730
1 - loss: 0.8784 - precision: 0.8521 - val_accuracy: 0.6577 - val_f1_score: 0.656
7 - val_loss: 1.2405 - val_precision: 0.7980
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7397 - f1_score: 0.7380
- loss: 0.8597 - precision: 0.8545
Epoch 8: val_loss did not improve from 1.22849
1314/1314 ————— 35s 26ms/step - accuracy: 0.7397 - f1_score: 0.737
9 - loss: 0.8597 - precision: 0.8544 - val_accuracy: 0.6583 - val_f1_score: 0.657
3 - val_loss: 1.2386 - val_precision: 0.7968
Epoch 9/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7391 - f1_score: 0.7372
- loss: 0.8583 - precision: 0.8555
Epoch 9: val_loss did not improve from 1.22849

1314/1314 ————— **35s** 26ms/step - accuracy: 0.7391 - f1_score: 0.737
 2 - loss: 0.8584 - precision: 0.8555 - val_accuracy: 0.6586 - val_f1_score: 0.657
 5 - val_loss: 1.2368 - val_precision: 0.7963
 Epoch 10/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.7400 - f1_score: 0.7385
 - loss: 0.8527 - precision: 0.8580
 Epoch 10: val_loss did not improve from 1.22849
1314/1314 ————— **35s** 26ms/step - accuracy: 0.7400 - f1_score: 0.738
 4 - loss: 0.8528 - precision: 0.8580 - val_accuracy: 0.6582 - val_f1_score: 0.656
 9 - val_loss: 1.2360 - val_precision: 0.7974
 Epoch 11/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.7424 - f1_score: 0.7406
 - loss: 0.8422 - precision: 0.8590
 Epoch 11: val_loss did not improve from 1.22849
1314/1314 ————— **35s** 26ms/step - accuracy: 0.7424 - f1_score: 0.740
 6 - loss: 0.8423 - precision: 0.8590 - val_accuracy: 0.6586 - val_f1_score: 0.657
 4 - val_loss: 1.2355 - val_precision: 0.7958
 Epoch 12/15
1312/1314 ————— **0s** 24ms/step - accuracy: 0.7466 - f1_score: 0.7449
 - loss: 0.8337 - precision: 0.8615
 Epoch 12: val_loss did not improve from 1.22849
1314/1314 ————— **35s** 26ms/step - accuracy: 0.7466 - f1_score: 0.744
 9 - loss: 0.8338 - precision: 0.8615 - val_accuracy: 0.6589 - val_f1_score: 0.657
 7 - val_loss: 1.2353 - val_precision: 0.7948
 Epoch 13/15
1314/1314 ————— **0s** 24ms/step - accuracy: 0.7492 - f1_score: 0.7473
 - loss: 0.8247 - precision: 0.8625
 Epoch 13: val_loss did not improve from 1.22849
1314/1314 ————— **35s** 26ms/step - accuracy: 0.7492 - f1_score: 0.747
 3 - loss: 0.8247 - precision: 0.8625 - val_accuracy: 0.6578 - val_f1_score: 0.656
 5 - val_loss: 1.2333 - val_precision: 0.7950
 Epoch 14/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.7477 - f1_score: 0.7460
 - loss: 0.8246 - precision: 0.8637
 Epoch 14: val_loss did not improve from 1.22849
1314/1314 ————— **35s** 26ms/step - accuracy: 0.7476 - f1_score: 0.745
 9 - loss: 0.8247 - precision: 0.8637 - val_accuracy: 0.6590 - val_f1_score: 0.657
 9 - val_loss: 1.2317 - val_precision: 0.7945
 Epoch 15/15
1313/1314 ————— **0s** 23ms/step - accuracy: 0.7491 - f1_score: 0.7474
 - loss: 0.8187 - precision: 0.8643
 Epoch 15: val_loss did not improve from 1.22849
1314/1314 ————— **34s** 26ms/step - accuracy: 0.7490 - f1_score: 0.747
 4 - loss: 0.8188 - precision: 0.8643 - val_accuracy: 0.6599 - val_f1_score: 0.658
 7 - val_loss: 1.2313 - val_precision: 0.7948



WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

141/141 ————— 10s 37ms/step - accuracy: 0.6536 - f1_score: 0.6462
- loss: 1.2442 - precision_9: 0.7832

Test Accuracy: 66.57%

Test Loss: 1.2198323011398315

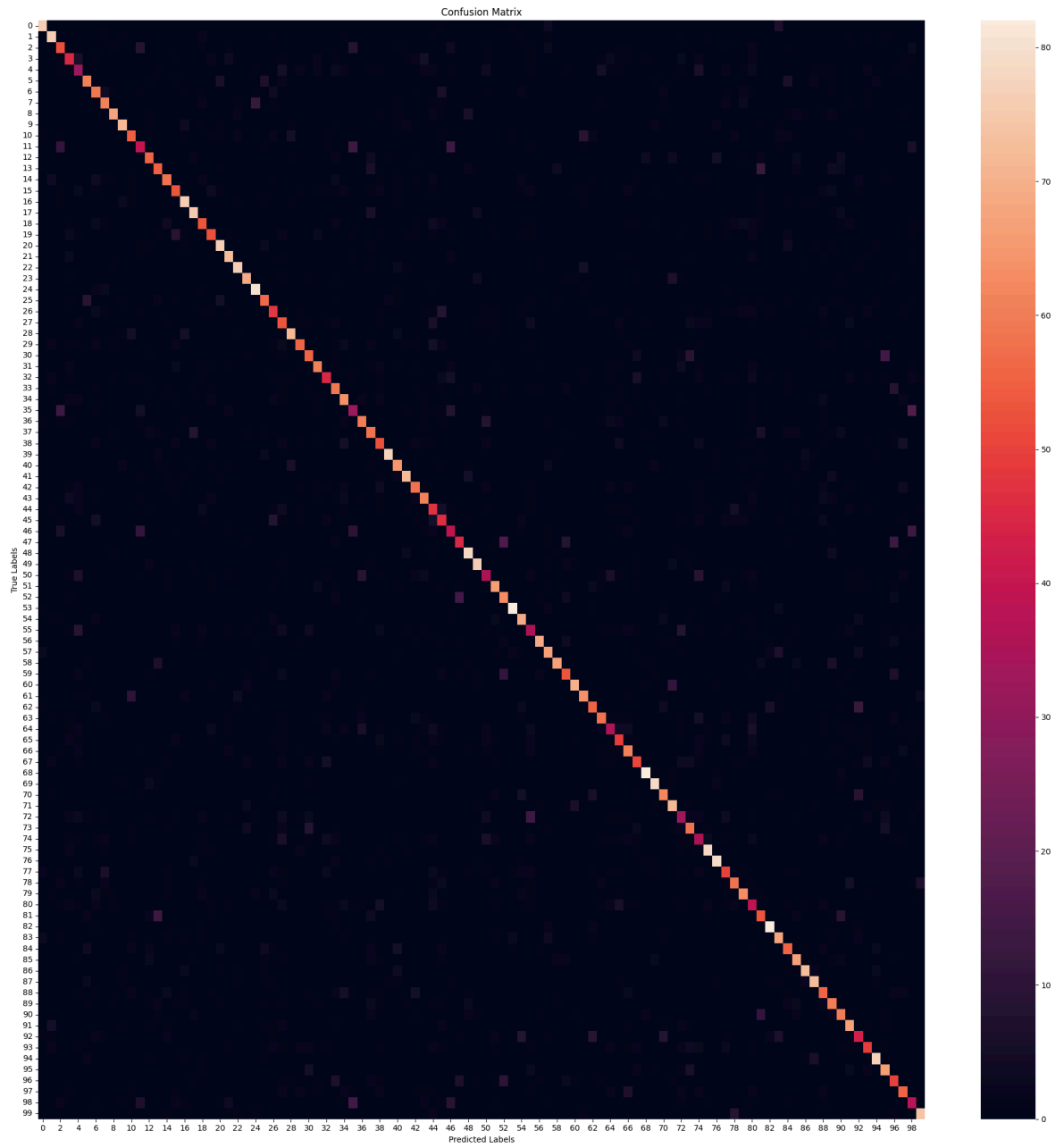
Test Precision: 79.64%

Test F1 Scores (Per Class): [84.91619 79.99999 55.319138 52.325577 35.555546 6
9.71428 61.458332

68.926544	80.473366	82.285706	63.953484	46.511627	64.36781	61.797745
67.83625	60.227264	79.58114	81.08108	60.919537	60.818714	80.85106
77.65957	83.51647	78.85714	83.76962	65.14285	53.93258	50.7177
80.89887	67.878784	63.636356	71.345024	55.214714	59.595955	68.0851
36.781605	68.965515	60.73298	56.216217	89.534874	68.783066	81.81818
65.16853	69.31818	46.766163	48.677242	43.575413	55.555553	81.24999
82.608696	42.424236	72.92817	64.94845	90.60773	71.50259	39.548016
81.6568	72.82608	80.23952	61.6279	81.39534	73.56321	64.36781
68.63905	45.283012	55.681812	64.210526	59.30232	89.13043	84.946236
69.27374	74.99999	37.647053	63.387978	39.325836	83.422455	85.86956
55.681812	67.44185	70.786514	41.052628	56.989246	87.70053	71.20418
62.92134	73.62636	81.56424	80.66298	61.538452	63.157898	63.82978
71.276596	49.42528	59.39393	85.55555	69.10995	53.191483	64.739876
42.45809	81.31868	]				

Average Test F1 Scores:66.48

282/282 ————— 11s 29ms/step



	precision	recall	f1-score	support
beaver	0.85	0.84	0.85	90
dolphin	0.76	0.84	0.80	90
otter	0.53	0.58	0.55	90
seal	0.55	0.50	0.52	90
whale	0.36	0.36	0.36	90
aquarium fish	0.72	0.68	0.70	90
flatfish	0.58	0.66	0.61	90
ray	0.69	0.68	0.69	90
shark	0.86	0.76	0.80	90
trout	0.85	0.80	0.82	90
orchids	0.67	0.61	0.64	90
poppies	0.49	0.44	0.47	90
roses	0.67	0.62	0.64	90
sunflowers	0.62	0.61	0.62	90
tulips	0.72	0.64	0.68	90
bottles	0.62	0.59	0.60	90
bowls	0.75	0.84	0.80	90
cans	0.79	0.83	0.81	90
cups	0.63	0.59	0.61	90
plates	0.64	0.58	0.61	90
apples	0.78	0.84	0.81	90
mushrooms	0.74	0.81	0.78	90
oranges	0.83	0.84	0.84	90
pears	0.81	0.77	0.79	90
sweet peppers	0.79	0.89	0.84	90
clock	0.67	0.63	0.65	90
computer keyboard	0.55	0.53	0.54	90
lamp	0.45	0.59	0.51	90
telephone	0.82	0.80	0.81	90
television	0.75	0.62	0.68	90
bed	0.65	0.62	0.64	90
chair	0.75	0.68	0.71	90
couch	0.62	0.50	0.55	90
table	0.55	0.66	0.60	90
wardrobe	0.65	0.71	0.68	90
bee	0.38	0.36	0.37	90
beetle	0.71	0.67	0.69	90
butterfly	0.57	0.64	0.61	90
caterpillar	0.55	0.58	0.56	90
cockroach	0.94	0.86	0.90	90
bear	0.66	0.72	0.69	90
leopard	0.84	0.80	0.82	90
lion	0.66	0.64	0.65	90
tiger	0.71	0.68	0.69	90
wolf	0.42	0.52	0.47	90
bridge	0.46	0.51	0.49	90
castle	0.44	0.43	0.44	90
house	0.62	0.50	0.56	90
road	0.76	0.87	0.81	90
skyscraper	0.81	0.84	0.83	90
cloud	0.47	0.39	0.42	90
forest	0.73	0.73	0.73	90
mountain	0.61	0.70	0.65	90
plain	0.90	0.91	0.91	90
sea	0.67	0.77	0.72	90
camel	0.40	0.39	0.40	90
cattle	0.87	0.77	0.82	90
chimpanzee	0.71	0.74	0.73	90

elephant	0.87	0.74	0.80	90
kangaroo	0.65	0.59	0.62	90
fox	0.85	0.78	0.81	90
porcupine	0.76	0.71	0.74	90
possum	0.67	0.62	0.65	90
raccoon	0.73	0.64	0.69	90
skunk	0.52	0.40	0.45	90
crab	0.57	0.54	0.56	90
lobster	0.61	0.68	0.64	90
snail	0.62	0.57	0.59	90
spider	0.87	0.91	0.89	90
worm	0.82	0.88	0.85	90
baby	0.70	0.69	0.69	90
boy	0.71	0.80	0.75	90
girl	0.40	0.36	0.38	90
man	0.62	0.64	0.63	90
woman	0.40	0.39	0.39	90
crocodile	0.80	0.87	0.83	90
dinosaur	0.84	0.88	0.86	90
lizard	0.57	0.56	0.56	90
snake	0.71	0.64	0.67	90
turtle	0.72	0.70	0.71	90
hamster	0.39	0.43	0.41	90
mouse	0.55	0.59	0.57	90
rabbit	0.85	0.91	0.88	90
shrew	0.67	0.76	0.71	90
squirrel	0.64	0.62	0.63	90
maple	0.73	0.74	0.74	90
oak	0.82	0.81	0.82	90
palm	0.80	0.81	0.81	90
pine	0.61	0.62	0.62	90
willow	0.60	0.67	0.63	90
bicycle	0.61	0.67	0.64	90
bus	0.68	0.74	0.71	90
motorcycle	0.51	0.48	0.49	90
pickup truck	0.65	0.54	0.59	90
train	0.86	0.86	0.86	90
lawn-mower	0.65	0.73	0.69	90
rocket	0.51	0.56	0.53	90
streetcar	0.67	0.62	0.65	90
tank	0.43	0.42	0.42	90
tractor	0.80	0.82	0.81	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.66	9000
weighted avg	0.67	0.67	0.66	9000

Dropout(0.6), L2Reg(3e-2), weight_decay(3e-2), train_augmentation

```
In [8]: for repeat_2_times in range(1):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

# .map(augment_dataset, num_parallel_calls=tf.data.experimental.
# .batch(batch_size)
# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-2) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'>
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model.h5', # Path where the best model will be saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

```

```

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model = load_model('best_model.h5')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>>

# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>

```

```

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[early_stopping_monitor])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune.h5')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores: {np.average(test_f1_scores.numpy()*100):.2f}%")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix, #cmap='Blues', fmt='d')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()

```



```
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

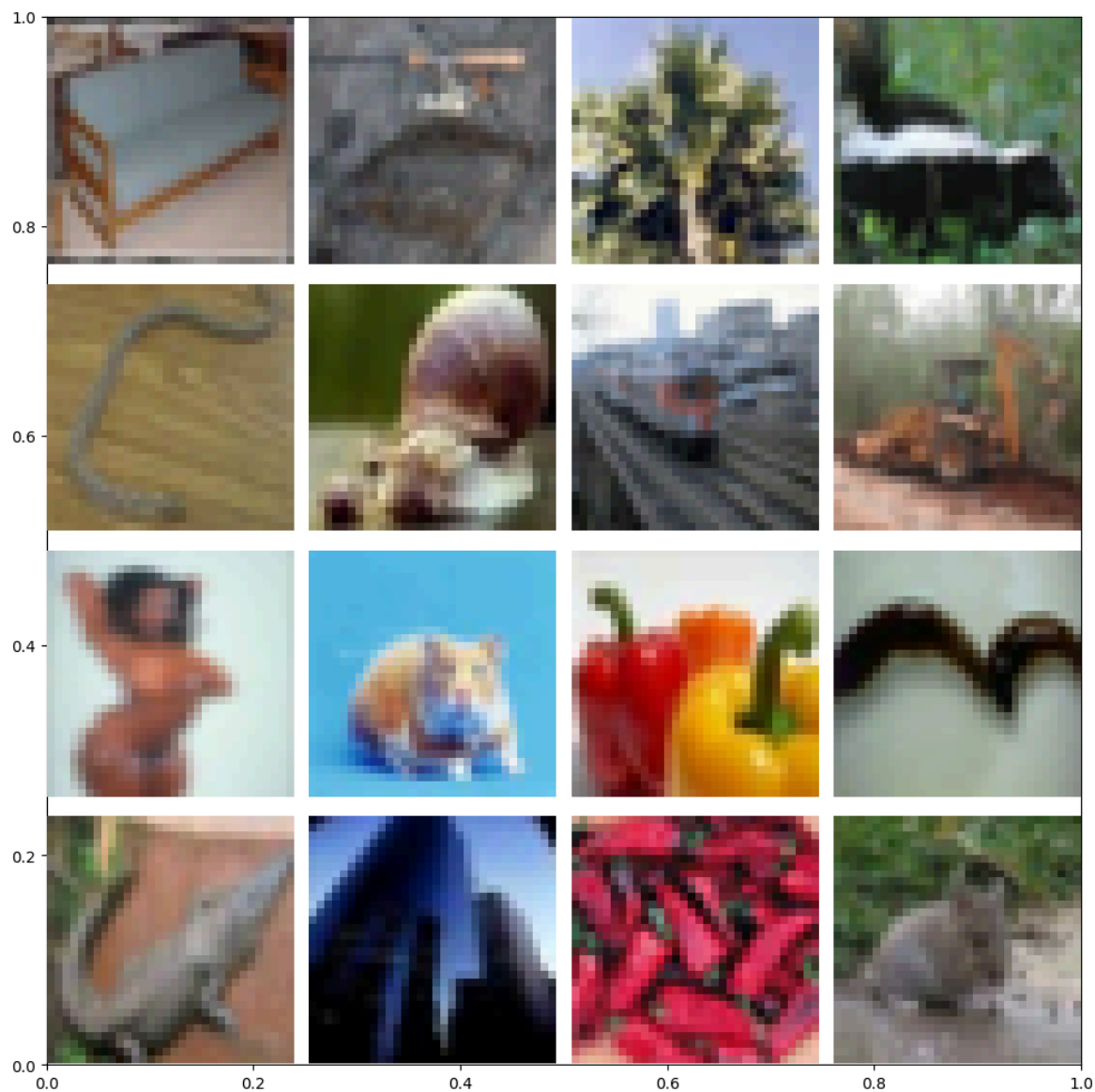
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

labels[13576].shape: (1,)

imgs[13576].shape: (32, 32, 3)

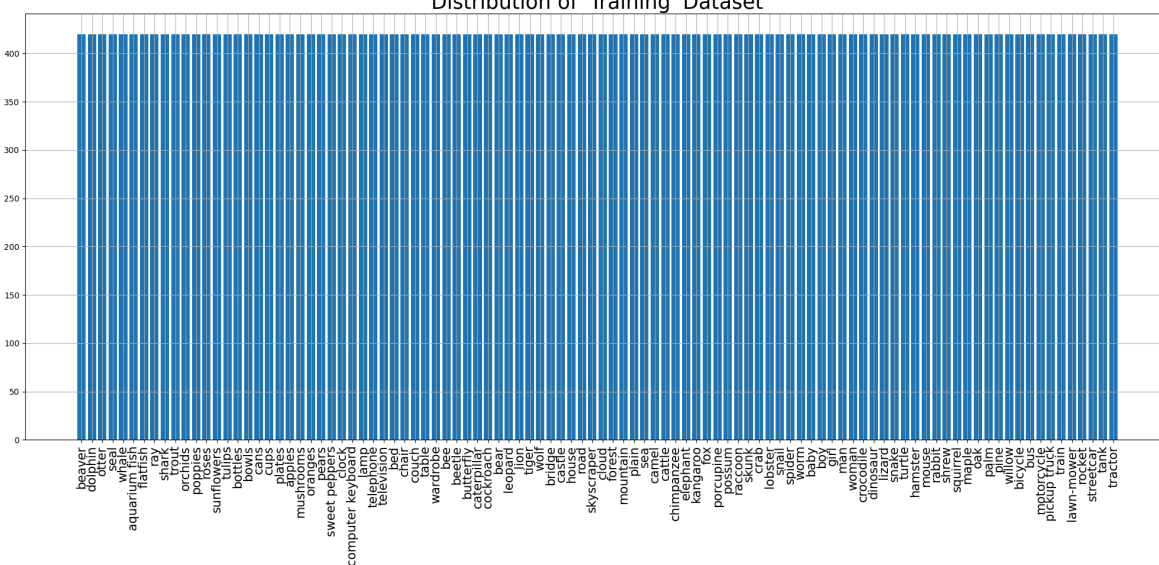


```
labels[29674].shape: (1,)
imgs[29674].shape: (32, 32, 3)
```

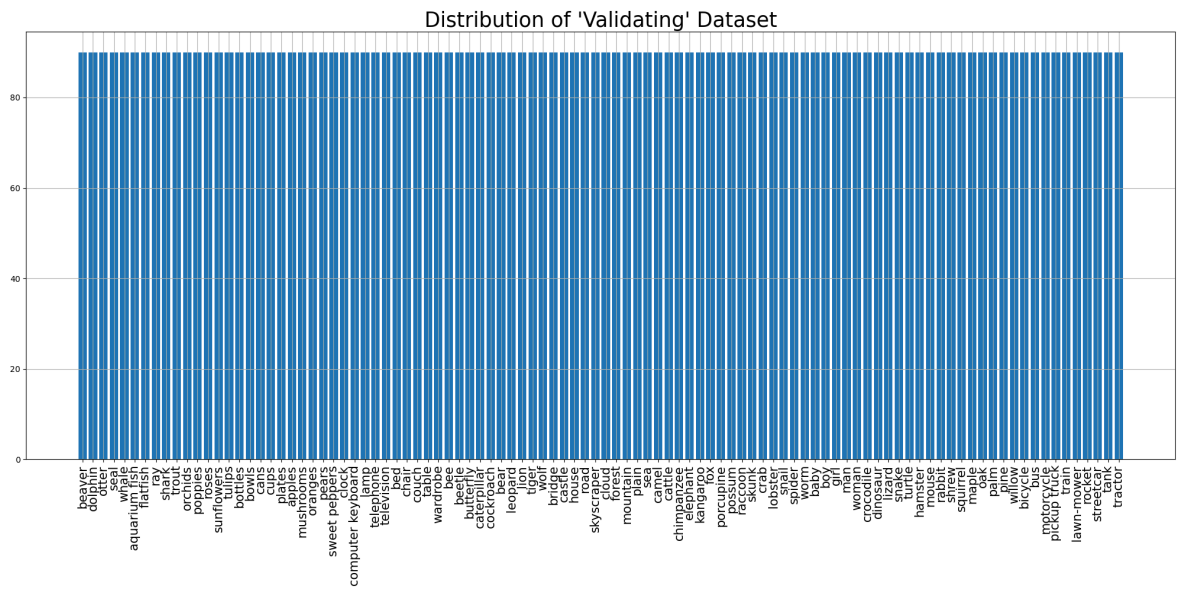


```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

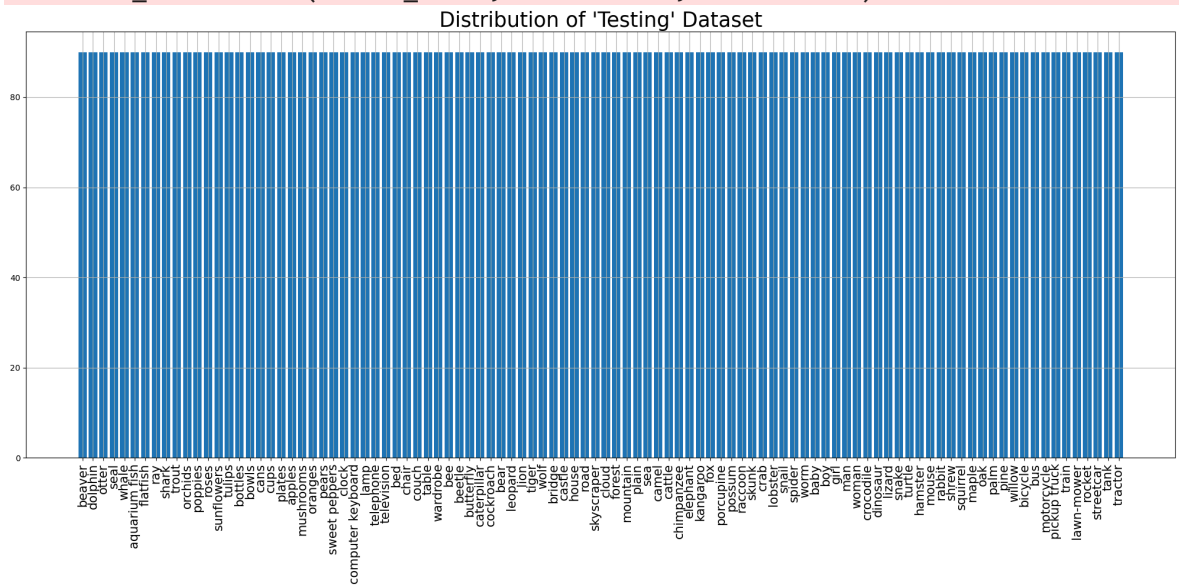
Distribution of 'Training' Dataset



```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels names, rotation=90, fontsize=15)
```



```
<ipython-input-4-d5e464050537>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)

Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/25

657/657 ————— 0s 29ms/step - accuracy: 0.3168 - f1_score: 0.3121 - loss: 2.9015 - precision: 0.6730

Epoch 1: val_loss improved from inf to 1.50837, saving model to best_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

657/657 ----- 40s 45ms/step - accuracy: 0.3169 - f1_score: 0.3123
- loss: 2.9006 - precision: 0.6731 - val_accuracy: 0.5899 - val_f1_score: 0.5862
- val_loss: 1.5084 - val_precision: 0.8240
Epoch 2/25
655/657 ----- 0s 23ms/step - accuracy: 0.5315 - f1_score: 0.5276 -
loss: 1.7204 - precision: 0.7434
Epoch 2: val_loss improved from 1.50837 to 1.38793, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5315 - f1_score: 0.5277
- loss: 1.7202 - precision: 0.7434 - val_accuracy: 0.6097 - val_f1_score: 0.6071
- val_loss: 1.3879 - val_precision: 0.8166
Epoch 3/25
657/657 ----- 0s 23ms/step - accuracy: 0.5655 - f1_score: 0.5624 -
loss: 1.5637 - precision: 0.7653
Epoch 3: val_loss improved from 1.38793 to 1.35741, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5655 - f1_score: 0.5624
- loss: 1.5636 - precision: 0.7653 - val_accuracy: 0.6156 - val_f1_score: 0.6131
- val_loss: 1.3574 - val_precision: 0.8080
Epoch 4/25
657/657 ----- 0s 23ms/step - accuracy: 0.5911 - f1_score: 0.5869 -
loss: 1.4391 - precision: 0.7761
Epoch 4: val_loss improved from 1.35741 to 1.34154, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
657/657 ----- 19s 29ms/step - accuracy: 0.5911 - f1_score: 0.5869
- loss: 1.4391 - precision: 0.7761 - val_accuracy: 0.6178 - val_f1_score: 0.6163
- val_loss: 1.3415 - val_precision: 0.7949
Epoch 5/25
656/657 ----- 0s 23ms/step - accuracy: 0.6096 - f1_score: 0.6072 -
loss: 1.3529 - precision: 0.7803
Epoch 5: val_loss improved from 1.34154 to 1.33329, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

#####
#####
657/657 ————— 19s 29ms/step - accuracy: 0.6096 - f1_score: 0.6072
- loss: 1.3529 - precision: 0.7803 - val_accuracy: 0.6218 - val_f1_score: 0.6211
- val_loss: 1.3333 - val_precision: 0.7923
Epoch 6/25
655/657 ————— 0s 24ms/step - accuracy: 0.6247 - f1_score: 0.6218 -
loss: 1.2953 - precision: 0.7880
Epoch 6: val_loss improved from 1.33329 to 1.31787, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
#####
#####
657/657 ————— 19s 29ms/step - accuracy: 0.6248 - f1_score: 0.6218
- loss: 1.2952 - precision: 0.7880 - val_accuracy: 0.6322 - val_f1_score: 0.6319
- val_loss: 1.3179 - val_precision: 0.7862
Epoch 7/25
655/657 ————— 0s 23ms/step - accuracy: 0.6390 - f1_score: 0.6361 -
loss: 1.2240 - precision: 0.7920
Epoch 7: val_loss improved from 1.31787 to 1.31158, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.

```

```

#####
#####

```

657/657 ————— 19s 29ms/step - accuracy: 0.6391 - f1_score: 0.6362
 - loss: 1.2239 - precision: 0.7920 - val_accuracy: 0.6292 - val_f1_score: 0.6291
 - val_loss: 1.3116 - val_precision: 0.7926

Epoch 8/25

655/657 ————— 0s 24ms/step - accuracy: 0.6594 - f1_score: 0.6569 -
 loss: 1.1643 - precision: 0.8033

Epoch 8: val_loss did not improve from 1.31158

657/657 ————— 19s 28ms/step - accuracy: 0.6594 - f1_score: 0.6569
 - loss: 1.1642 - precision: 0.8032 - val_accuracy: 0.6361 - val_f1_score: 0.6342
 - val_loss: 1.3125 - val_precision: 0.7848

Epoch 9/25

656/657 ————— 0s 23ms/step - accuracy: 0.6639 - f1_score: 0.6616 -
 loss: 1.1284 - precision: 0.8042

Epoch 9: val_loss did not improve from 1.31158

657/657 ————— 18s 28ms/step - accuracy: 0.6639 - f1_score: 0.6616
 - loss: 1.1283 - precision: 0.8042 - val_accuracy: 0.6320 - val_f1_score: 0.6304
 - val_loss: 1.3360 - val_precision: 0.7806

Epoch 10/25

656/657 ————— 0s 23ms/step - accuracy: 0.6813 - f1_score: 0.6790 -
 loss: 1.0711 - precision: 0.8140

Epoch 10: val_loss did not improve from 1.31158

657/657 ————— 18s 28ms/step - accuracy: 0.6813 - f1_score: 0.6790
 - loss: 1.0711 - precision: 0.8140 - val_accuracy: 0.6346 - val_f1_score: 0.6336
 - val_loss: 1.3212 - val_precision: 0.7761

Epoch 11/25

655/657 ————— 0s 24ms/step - accuracy: 0.6879 - f1_score: 0.6857 -
 loss: 1.0291 - precision: 0.8152

Epoch 11: val_loss did not improve from 1.31158

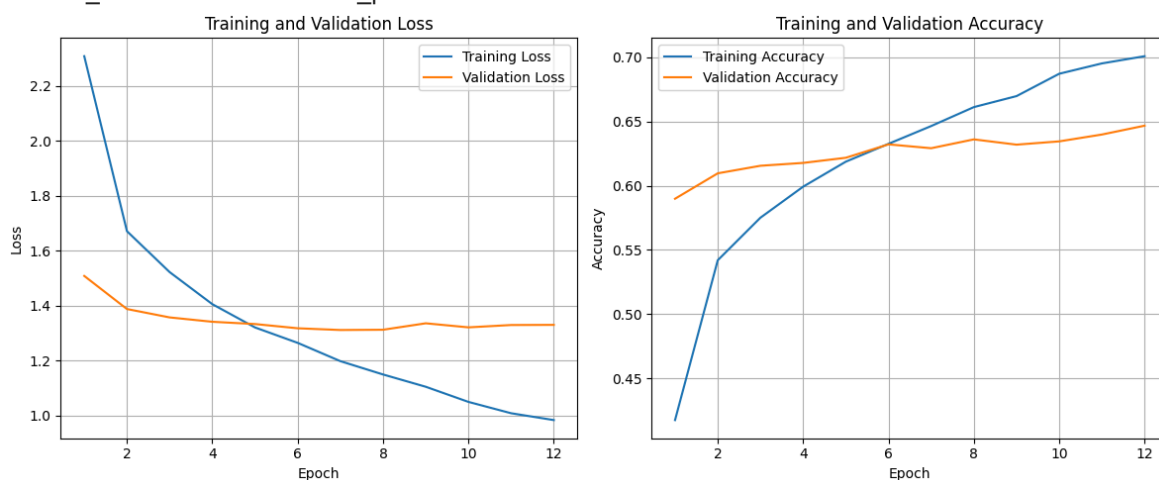
657/657 ————— 19s 29ms/step - accuracy: 0.6879 - f1_score: 0.6857
 - loss: 1.0290 - precision: 0.8152 - val_accuracy: 0.6399 - val_f1_score: 0.6392
 - val_loss: 1.3297 - val_precision: 0.7771

Epoch 12/25

656/657 ————— 0s 24ms/step - accuracy: 0.6963 - f1_score: 0.6943 -
 loss: 0.9981 - precision: 0.8186

Epoch 12: val_loss did not improve from 1.31158

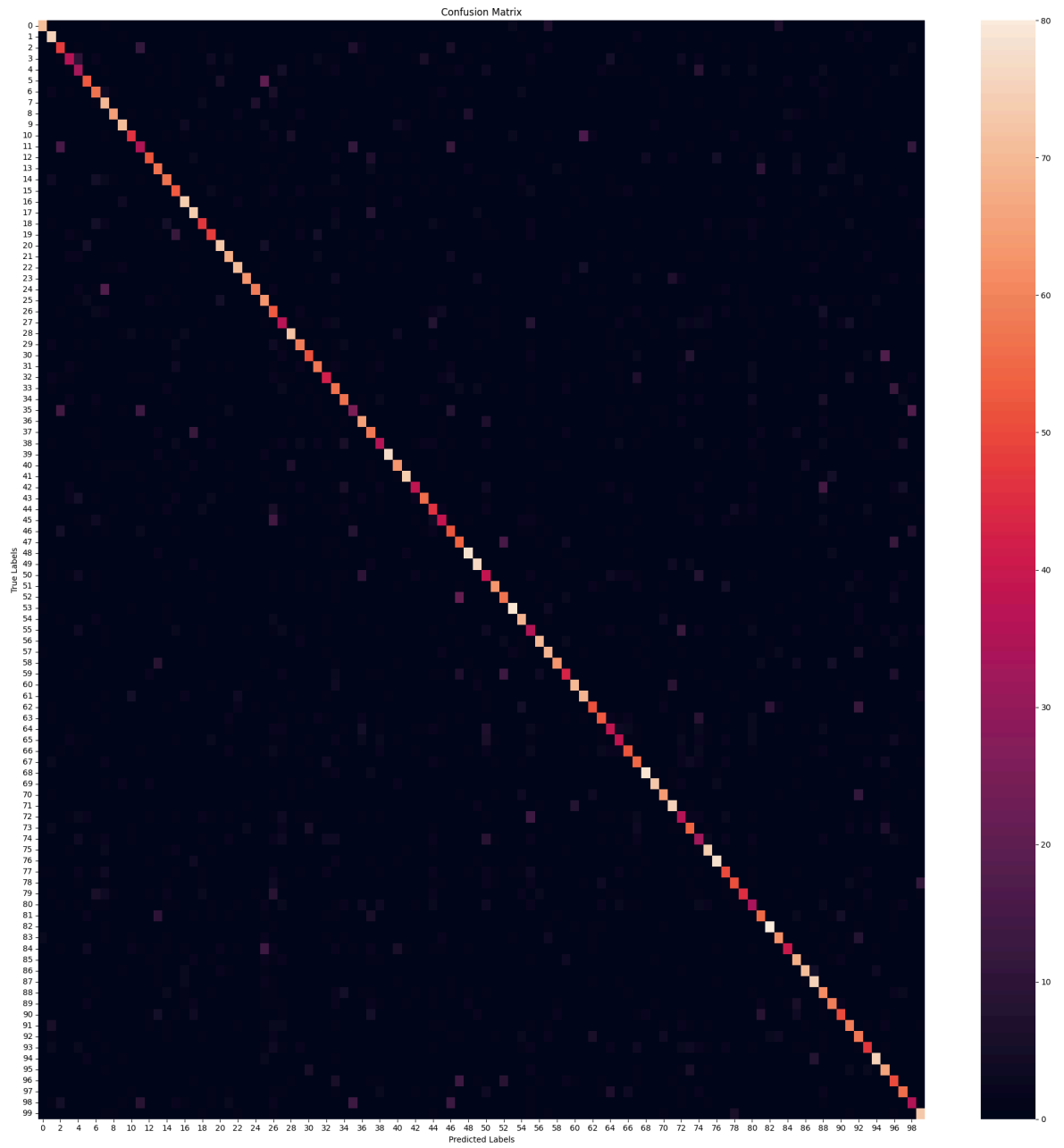
657/657 ————— 19s 28ms/step - accuracy: 0.6963 - f1_score: 0.6944
 - loss: 0.9981 - precision: 0.8186 - val_accuracy: 0.6468 - val_f1_score: 0.6458
 - val_loss: 1.3303 - val_precision: 0.7759



WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

141/141  **10s** 37ms/step - accuracy: 0.6315 - f1_score: 0.6262
- loss: 1.3122 - precision_11: 0.7931
Test Accuracy: 63.80%
Test Loss: 1.2928075790405273
Test Precision: 80.02%
Test F1 Scores (Per Class): [85.02993 73.891624 52.173912 48.32214 37.714283 62.352936 59.685856 62.780266 80.24691 80.681816 59.740257 41.916164 65. 61.62162 64.40677 56.684486 78.723404 75.12689 58.385086 57.142853 80.66298 73.91304 80.22598 78.75 74.68353 56.502235 43.442616 41.111107 77.83783 64.480865 63.354034 68.23529 51.851852 51.81818 62.63736 31.645563 68.42105 57.425735 44.72049 91.12425 69.23077 85.057465 53.14685 69.99999 47.91666 49.681526 51.74129 55.384613 82.10526 81.28342 39.195972 75.90361 60.63829 88.26815 72.63157 35.71428 80.459755 73.01586 77.300606 53.987724 80.92485 73.40425 63.354034 62.72189 46.060596 48.407635 60.465115 56.994812 87.77777 84.39305 73.14285 77.7202 38.50267 59.999996 33.160614 79.569885 84.7826 56.818176 64.15094 61.643826 36.55913 61.111103 86.02149 70.85714 55.555546 66.34146 81.81818 77.31957 53.2751 66.666664 60.975605 63.440853 55.76923 54.02298 84.26965 65.346535 48.543686 61.538452 41.176464 78.49462]
Average Test F1 Scores:63.90

282/282  **10s** 26ms/step



	precision	recall	f1-score	support
beaver	0.92	0.79	0.85	90
dolphin	0.66	0.83	0.74	90
otter	0.51	0.53	0.52	90
seal	0.62	0.40	0.49	90
whale	0.39	0.37	0.38	90
aquarium fish	0.66	0.59	0.62	90
flatfish	0.56	0.63	0.60	90
ray	0.53	0.78	0.63	90
shark	0.90	0.72	0.80	90
trout	0.83	0.79	0.81	90
orchids	0.72	0.51	0.60	90
poppies	0.45	0.39	0.42	90
roses	0.74	0.58	0.65	90
sunflowers	0.60	0.63	0.62	90
tulips	0.66	0.63	0.64	90
bottles	0.55	0.59	0.57	90
bowls	0.76	0.82	0.79	90
cans	0.69	0.82	0.75	90
cups	0.66	0.52	0.58	90
plates	0.61	0.53	0.57	90
apples	0.80	0.81	0.81	90
mushrooms	0.72	0.76	0.74	90
oranges	0.82	0.79	0.80	90
pears	0.90	0.70	0.79	90
sweet peppers	0.87	0.66	0.75	90
clock	0.47	0.70	0.57	90
computer keyboard	0.34	0.59	0.43	90
lamp	0.41	0.41	0.41	90
telephone	0.76	0.80	0.78	90
television	0.63	0.66	0.64	90
bed	0.72	0.57	0.63	90
chair	0.72	0.64	0.68	90
couch	0.58	0.47	0.52	90
table	0.44	0.63	0.52	90
wardrobe	0.62	0.63	0.63	90
bee	0.37	0.28	0.32	90
beetle	0.65	0.72	0.68	90
butterfly	0.51	0.64	0.57	90
caterpillar	0.51	0.40	0.45	90
cockroach	0.97	0.86	0.91	90
bear	0.68	0.70	0.69	90
leopard	0.88	0.82	0.85	90
lion	0.72	0.42	0.53	90
tiger	0.80	0.62	0.70	90
wolf	0.45	0.51	0.48	90
bridge	0.58	0.43	0.50	90
castle	0.47	0.58	0.52	90
house	0.51	0.60	0.55	90
road	0.78	0.87	0.82	90
skyscraper	0.78	0.84	0.81	90
cloud	0.36	0.43	0.39	90
forest	0.83	0.70	0.76	90
mountain	0.58	0.63	0.61	90
plain	0.89	0.88	0.88	90
sea	0.69	0.77	0.73	90
camel	0.33	0.39	0.36	90
cattle	0.83	0.78	0.80	90
chimpanzee	0.70	0.77	0.73	90

elephant	0.86	0.70	0.77	90
kangaroo	0.60	0.48	0.53	90
fox	0.84	0.78	0.81	90
porcupine	0.70	0.77	0.73	90
possum	0.72	0.57	0.63	90
raccoon	0.67	0.59	0.63	90
skunk	0.51	0.42	0.46	90
crab	0.57	0.42	0.48	90
lobster	0.63	0.58	0.60	90
snail	0.53	0.61	0.57	90
spider	0.88	0.88	0.88	90
worm	0.88	0.81	0.84	90
baby	0.75	0.71	0.73	90
boy	0.73	0.83	0.78	90
girl	0.37	0.40	0.39	90
man	0.60	0.60	0.60	90
woman	0.31	0.36	0.33	90
crocodile	0.77	0.82	0.80	90
dinosaur	0.83	0.87	0.85	90
lizard	0.58	0.56	0.57	90
snake	0.74	0.57	0.64	90
turtle	0.80	0.50	0.62	90
hamster	0.35	0.38	0.37	90
mouse	0.61	0.61	0.61	90
rabbit	0.83	0.89	0.86	90
shrew	0.73	0.69	0.71	90
squirrel	0.74	0.44	0.56	90
maple	0.59	0.76	0.66	90
oak	0.84	0.79	0.81	90
palm	0.72	0.83	0.77	90
pine	0.44	0.68	0.53	90
willow	0.68	0.66	0.67	90
bicycle	0.68	0.56	0.61	90
bus	0.61	0.66	0.63	90
motorcycle	0.49	0.64	0.56	90
pickup truck	0.56	0.52	0.54	90
train	0.85	0.83	0.84	90
lawn-mower	0.59	0.73	0.65	90
rocket	0.43	0.56	0.49	90
streetcar	0.61	0.62	0.62	90
tank	0.44	0.39	0.41	90
tractor	0.76	0.81	0.78	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



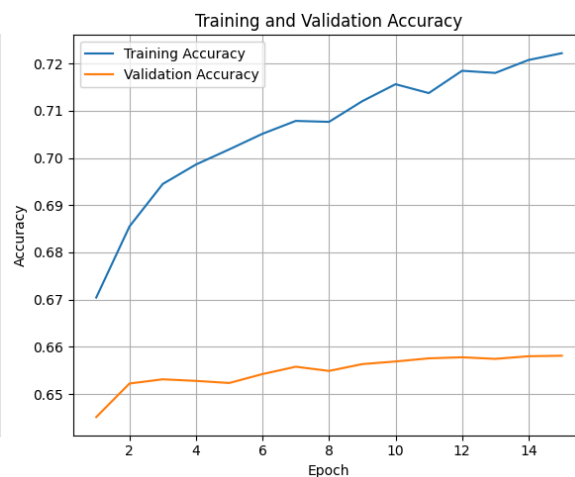
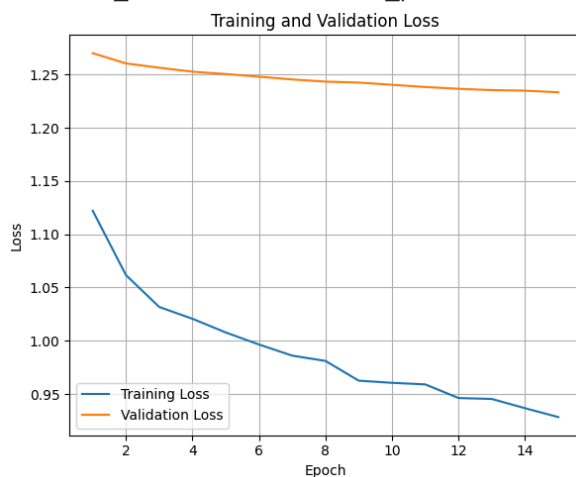
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 26ms/step - accuracy: 0.6687 - f1_score: 0.6673
- loss: 1.1198 - precision: 0.8144
Epoch 1: val_loss did not improve from 1.22849
1314/1314 ————— 54s 33ms/step - accuracy: 0.6687 - f1_score: 0.6673
- loss: 1.1198 - precision: 0.8144 - val_accuracy: 0.6451 - val_f1_score: 0.6439
- val_loss: 1.2701 - val_precision: 0.7937
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6870 - f1_score: 0.6848
- loss: 1.0438 - precision: 0.8255
Epoch 2: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.6870 - f1_score: 0.6848
- loss: 1.0438 - precision: 0.8255 - val_accuracy: 0.6522 - val_f1_score: 0.6511
- val_loss: 1.2605 - val_precision: 0.7926
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7005 - f1_score: 0.6985
- loss: 1.0071 - precision: 0.8345
Epoch 3: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7005 - f1_score: 0.6984
- loss: 1.0072 - precision: 0.8345 - val_accuracy: 0.6531 - val_f1_score: 0.6520
- val_loss: 1.2565 - val_precision: 0.7908
Epoch 4/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.7036 - f1_score: 0.7018
- loss: 0.9961 - precision: 0.8367
Epoch 4: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7036 - f1_score: 0.7018
- loss: 0.9961 - precision: 0.8367 - val_accuracy: 0.6528 - val_f1_score: 0.6519
- val_loss: 1.2528 - val_precision: 0.7930
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7077 - f1_score: 0.7057
- loss: 0.9833 - precision: 0.8387
Epoch 5: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7077 - f1_score: 0.7057
- loss: 0.9833 - precision: 0.8387 - val_accuracy: 0.6523 - val_f1_score: 0.6515
- val_loss: 1.2505 - val_precision: 0.7936
Epoch 6/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7121 - f1_score: 0.7099
- loss: 0.9687 - precision: 0.8421
Epoch 6: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7120 - f1_score: 0.7099
- loss: 0.9688 - precision: 0.8421 - val_accuracy: 0.6542 - val_f1_score: 0.6532
- val_loss: 1.2481 - val_precision: 0.7942
Epoch 7/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7128 - f1_score: 0.7109
- loss: 0.9573 - precision: 0.8433
Epoch 7: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7128 - f1_score: 0.7109
- loss: 0.9574 - precision: 0.8433 - val_accuracy: 0.6558 - val_f1_score: 0.6547
- val_loss: 1.2455 - val_precision: 0.7952
Epoch 8/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.7144 - f1_score: 0.7125
- loss: 0.9489 - precision: 0.8456
Epoch 8: val_loss did not improve from 1.22849
1314/1314 ————— 34s 26ms/step - accuracy: 0.7144 - f1_score: 0.7125
- loss: 0.9490 - precision: 0.8456 - val_accuracy: 0.6549 - val_f1_score: 0.6539
- val_loss: 1.2434 - val_precision: 0.7961
Epoch 9/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7211 - f1_score: 0.7194
- loss: 0.9290 - precision: 0.8515
Epoch 9: val_loss did not improve from 1.22849

1314/1314 — 34s 26ms/step - accuracy: 0.7211 - f1_score: 0.719
 4 - loss: 0.9290 - precision: 0.8515 - val_accuracy: 0.6563 - val_f1_score: 0.655
 6 - val_loss: 1.2425 - val_precision: 0.7930
 Epoch 10/15
 1314/1314 — 0s 24ms/step - accuracy: 0.7226 - f1_score: 0.7206
 - loss: 0.9282 - precision: 0.8504
 Epoch 10: val_loss did not improve from 1.22849
 1314/1314 — 34s 26ms/step - accuracy: 0.7226 - f1_score: 0.720
 6 - loss: 0.9283 - precision: 0.8504 - val_accuracy: 0.6569 - val_f1_score: 0.655
 8 - val_loss: 1.2405 - val_precision: 0.7950
 Epoch 11/15
 1314/1314 — 0s 24ms/step - accuracy: 0.7212 - f1_score: 0.7195
 - loss: 0.9257 - precision: 0.8481
 Epoch 11: val_loss did not improve from 1.22849
 1314/1314 — 34s 26ms/step - accuracy: 0.7212 - f1_score: 0.719
 5 - loss: 0.9257 - precision: 0.8481 - val_accuracy: 0.6576 - val_f1_score: 0.656
 7 - val_loss: 1.2384 - val_precision: 0.7954
 Epoch 12/15
 1312/1314 — 0s 23ms/step - accuracy: 0.7257 - f1_score: 0.7241
 - loss: 0.9087 - precision: 0.8532
 Epoch 12: val_loss did not improve from 1.22849
 1314/1314 — 34s 26ms/step - accuracy: 0.7257 - f1_score: 0.724
 0 - loss: 0.9087 - precision: 0.8532 - val_accuracy: 0.6578 - val_f1_score: 0.656
 8 - val_loss: 1.2367 - val_precision: 0.7955
 Epoch 13/15
 1313/1314 — 0s 24ms/step - accuracy: 0.7249 - f1_score: 0.7230
 - loss: 0.9118 - precision: 0.8531
 Epoch 13: val_loss did not improve from 1.22849
 1314/1314 — 34s 26ms/step - accuracy: 0.7249 - f1_score: 0.723
 0 - loss: 0.9118 - precision: 0.8531 - val_accuracy: 0.6574 - val_f1_score: 0.656
 3 - val_loss: 1.2355 - val_precision: 0.7948
 Epoch 14/15
 1313/1314 — 0s 24ms/step - accuracy: 0.7285 - f1_score: 0.7266
 - loss: 0.8996 - precision: 0.8551
 Epoch 14: val_loss did not improve from 1.22849
 1314/1314 — 34s 26ms/step - accuracy: 0.7285 - f1_score: 0.726
 6 - loss: 0.8997 - precision: 0.8551 - val_accuracy: 0.6580 - val_f1_score: 0.657
 0 - val_loss: 1.2349 - val_precision: 0.7935
 Epoch 15/15
 1313/1314 — 0s 24ms/step - accuracy: 0.7291 - f1_score: 0.7274
 - loss: 0.8922 - precision: 0.8544
 Epoch 15: val_loss did not improve from 1.22849
 1314/1314 — 34s 26ms/step - accuracy: 0.7291 - f1_score: 0.727
 4 - loss: 0.8922 - precision: 0.8544 - val_accuracy: 0.6581 - val_f1_score: 0.657
 2 - val_loss: 1.2334 - val_precision: 0.7935



WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

141/141 ————— 11s 37ms/step - accuracy: 0.6536 - f1_score: 0.6462
- loss: 1.2442 - precision_13: 0.7832

Test Accuracy: 66.57%

Test Loss: 1.2198323011398315

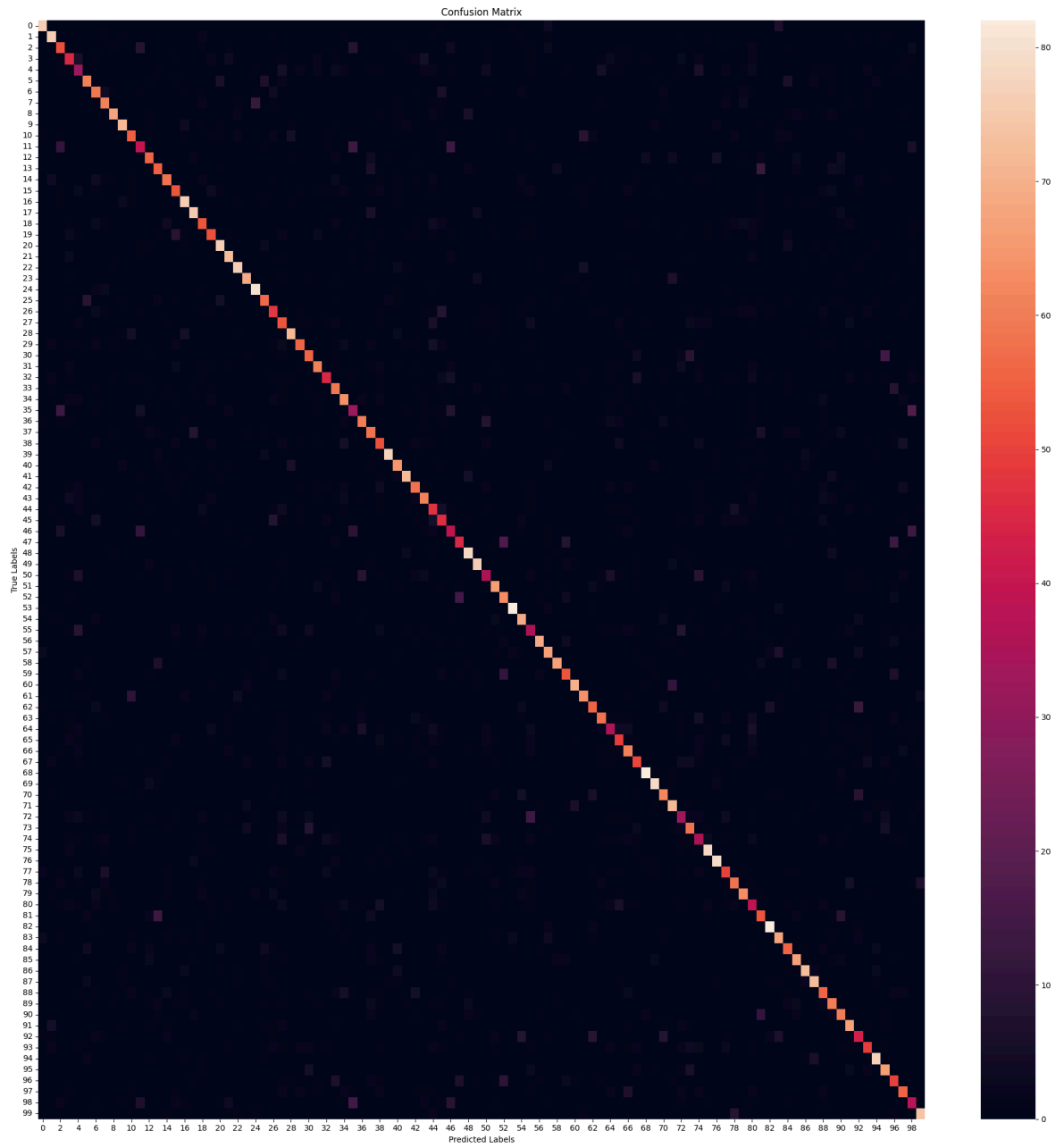
Test Precision: 79.64%

Test F1 Scores (Per Class): [84.91619 79.99999 55.319138 52.325577 35.555546 6
9.71428 61.458332

68.926544	80.473366	82.285706	63.953484	46.511627	64.36781	61.797745
67.83625	60.227264	79.58114	81.08108	60.919537	60.818714	80.85106
77.65957	83.51647	78.85714	83.76962	65.14285	53.93258	50.7177
80.89887	67.878784	63.636356	71.345024	55.214714	59.595955	68.0851
36.781605	68.965515	60.73298	56.216217	89.534874	68.783066	81.81818
65.16853	69.31818	46.766163	48.677242	43.575413	55.555553	81.24999
82.608696	42.424236	72.92817	64.94845	90.60773	71.50259	39.548016
81.6568	72.82608	80.23952	61.6279	81.39534	73.56321	64.36781
68.63905	45.283012	55.681812	64.210526	59.30232	89.13043	84.946236
69.27374	74.99999	37.647053	63.387978	39.325836	83.422455	85.86956
55.681812	67.44185	70.786514	41.052628	56.989246	87.70053	71.20418
62.92134	73.62636	81.56424	80.66298	61.538452	63.157898	63.82978
71.276596	49.42528	59.39393	85.55555	69.10995	53.191483	64.739876
42.45809	81.31868	]				

Average Test F1 Scores:66.48

282/282 ————— 11s 26ms/step



	precision	recall	f1-score	support
beaver	0.85	0.84	0.85	90
dolphin	0.76	0.84	0.80	90
otter	0.53	0.58	0.55	90
seal	0.55	0.50	0.52	90
whale	0.36	0.36	0.36	90
aquarium fish	0.72	0.68	0.70	90
flatfish	0.58	0.66	0.61	90
ray	0.69	0.68	0.69	90
shark	0.86	0.76	0.80	90
trout	0.85	0.80	0.82	90
orchids	0.67	0.61	0.64	90
poppies	0.49	0.44	0.47	90
roses	0.67	0.62	0.64	90
sunflowers	0.62	0.61	0.62	90
tulips	0.72	0.64	0.68	90
bottles	0.62	0.59	0.60	90
bowls	0.75	0.84	0.80	90
cans	0.79	0.83	0.81	90
cups	0.63	0.59	0.61	90
plates	0.64	0.58	0.61	90
apples	0.78	0.84	0.81	90
mushrooms	0.74	0.81	0.78	90
oranges	0.83	0.84	0.84	90
pears	0.81	0.77	0.79	90
sweet peppers	0.79	0.89	0.84	90
clock	0.67	0.63	0.65	90
computer keyboard	0.55	0.53	0.54	90
lamp	0.45	0.59	0.51	90
telephone	0.82	0.80	0.81	90
television	0.75	0.62	0.68	90
bed	0.65	0.62	0.64	90
chair	0.75	0.68	0.71	90
couch	0.62	0.50	0.55	90
table	0.55	0.66	0.60	90
wardrobe	0.65	0.71	0.68	90
bee	0.38	0.36	0.37	90
beetle	0.71	0.67	0.69	90
butterfly	0.57	0.64	0.61	90
caterpillar	0.55	0.58	0.56	90
cockroach	0.94	0.86	0.90	90
bear	0.66	0.72	0.69	90
leopard	0.84	0.80	0.82	90
lion	0.66	0.64	0.65	90
tiger	0.71	0.68	0.69	90
wolf	0.42	0.52	0.47	90
bridge	0.46	0.51	0.49	90
castle	0.44	0.43	0.44	90
house	0.62	0.50	0.56	90
road	0.76	0.87	0.81	90
skyscraper	0.81	0.84	0.83	90
cloud	0.47	0.39	0.42	90
forest	0.73	0.73	0.73	90
mountain	0.61	0.70	0.65	90
plain	0.90	0.91	0.91	90
sea	0.67	0.77	0.72	90
camel	0.40	0.39	0.40	90
cattle	0.87	0.77	0.82	90
chimpanzee	0.71	0.74	0.73	90

elephant	0.87	0.74	0.80	90
kangaroo	0.65	0.59	0.62	90
fox	0.85	0.78	0.81	90
porcupine	0.76	0.71	0.74	90
possum	0.67	0.62	0.65	90
raccoon	0.73	0.64	0.69	90
skunk	0.52	0.40	0.45	90
crab	0.57	0.54	0.56	90
lobster	0.61	0.68	0.64	90
snail	0.62	0.57	0.59	90
spider	0.87	0.91	0.89	90
worm	0.82	0.88	0.85	90
baby	0.70	0.69	0.69	90
boy	0.71	0.80	0.75	90
girl	0.40	0.36	0.38	90
man	0.62	0.64	0.63	90
woman	0.40	0.39	0.39	90
crocodile	0.80	0.87	0.83	90
dinosaur	0.84	0.88	0.86	90
lizard	0.57	0.56	0.56	90
snake	0.71	0.64	0.67	90
turtle	0.72	0.70	0.71	90
hamster	0.39	0.43	0.41	90
mouse	0.55	0.59	0.57	90
rabbit	0.85	0.91	0.88	90
shrew	0.67	0.76	0.71	90
squirrel	0.64	0.62	0.63	90
maple	0.73	0.74	0.74	90
oak	0.82	0.81	0.82	90
palm	0.80	0.81	0.81	90
pine	0.61	0.62	0.62	90
willow	0.60	0.67	0.63	90
bicycle	0.61	0.67	0.64	90
bus	0.68	0.74	0.71	90
motorcycle	0.51	0.48	0.49	90
pickup truck	0.65	0.54	0.59	90
train	0.86	0.86	0.86	90
lawn-mower	0.65	0.73	0.69	90
rocket	0.51	0.56	0.53	90
streetcar	0.67	0.62	0.65	90
tank	0.43	0.42	0.42	90
tractor	0.80	0.82	0.81	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.66	9000
weighted avg	0.67	0.67	0.66	9000

In []: